

# Contents

|                           |           |
|---------------------------|-----------|
| <b>字串操作函式參考</b>           | <b>4</b>  |
| 函式一覽                      | 4         |
| contains                  | 5         |
| starts_with               | 5         |
| ends_with                 | 5         |
| find                      | 6         |
| substring                 | 6         |
| char_at                   | 6         |
| replace                   | 6         |
| to_upper                  | 6         |
| to_lower                  | 6         |
| trim                      | 7         |
| trim_start                | 7         |
| trim_end                  | 7         |
| repeat                    | 7         |
| reverse                   | 7         |
| byte_len                  | 8         |
| split                     | 8         |
| join                      | 8         |
| to_int                    | 8         |
| to_float                  | 8         |
| to_str                    | 9         |
| <b>內建函式參考</b>             | <b>9</b>  |
| 函式一覽                      | 9         |
| print                     | 10        |
| Some                      | 11        |
| len                       | 11        |
| has_key                   | 11        |
| add                       | 12        |
| remove                    | 12        |
| range                     | 12        |
| exit                      | 13        |
| args                      | 13        |
| append                    | 13        |
| pop                       | 13        |
| reverse (串列)              | 13        |
| slice                     | 14        |
| filter                    | 14        |
| map                       | 14        |
| sort                      | 14        |
| sort!                     | 15        |
| reverse!                  | 15        |
| appended                  | 15        |
| append!                   | 15        |
| first                     | 15        |
| last                      | 15        |
| get (映射)                  | 16        |
| <b>集合參考 (元組、串列、映射、集合)</b> | <b>16</b> |
| 元組                        | 16        |
| 串列                        | 17        |
| 映射                        | 22        |

|                                   |           |
|-----------------------------------|-----------|
| 集合                                | 23        |
| <b>契約式設計 (Design by Contract)</b> | <b>25</b> |
| 前置條件 (require)                    | 26        |
| 後置條件 (ensure)                     | 26        |
| 組合範例                              | 26        |
| 結構體不變量 (invariant)                | 27        |
| 規則                                | 27        |
| <b>控制流程參考</b>                     | <b>27</b> |
| if / elif / else                  | 27        |
| while                             | 28        |
| for                               | 28        |
| break                             | 30        |
| continue                          | 30        |
| ... (Ellipsis)                    | 30        |
| match                             | 30        |
| 作用域規則                             | 33        |
| <b>指令</b>                         | <b>33</b> |
| 語法                                | 33        |
| 支援的目標                             | 33        |
| 內建指令                              | 34        |
| 注意事項                              | 35        |
| <b>錯誤參考</b>                       | <b>36</b> |
| 錯誤格式                              | 36        |
| 編譯錯誤一覽                            | 36        |
| 執行時錯誤一覽                           | 37        |
| <b>函式參考</b>                       | <b>38</b> |
| 函式定義語法                            | 38        |
| 引數與回傳值的型別                         | 38        |
| 遞迴                                | 38        |
| 多載                                | 38        |
| Unit 型別函式                         | 39        |
| Lambda 函式                         | 39        |
| 閉包                                | 40        |
| 函式型別                              | 40        |
| 高階函式                              | 40        |
| UFCS (統一函式呼叫語法)                   | 41        |
| 運算子多載                             | 41        |
| <b>運算子參考</b>                      | <b>42</b> |
| 優先順序表                             | 42        |
| 算術運算子                             | 42        |
| 比較運算子                             | 43        |
| 邏輯運算子                             | 43        |
| 位元運算子                             | 44        |
| 三元條件運算子                           | 44        |
| 範圍運算子                             | 44        |
| 空值合併運算子 (??)                      | 44        |
| 複合賦值運算子                           | 45        |
| 遞增・遞減運算子                          | 45        |
| 運算的型別規則                           | 45        |

|                       |           |
|-----------------------|-----------|
| 運算子多載                 | 46        |
| <b>套件參考</b>           | <b>47</b> |
| 概述                    | 47        |
| 匯入語法                  | 47        |
| 套件解析                  | 47        |
| 標準函式庫 (std)           | 48        |
| 搜尋路徑的優先順序             | 48        |
| RY_PATH 環境變數          | 48        |
| 限制                    | 48        |
| 建立套件檔案                | 48        |
| <b>專案管理</b>           | <b>49</b> |
| ry init - 專案初始化       | 49        |
| ry self-update - 自我更新 | 49        |
| ry.toml 設定檔           | 50        |
| <b>正規表達式函式參考手冊</b>    | <b>50</b> |
| 函式一覽                  | 51        |
| 支援的模式語法               | 51        |
| 使用範例                  | 51        |
| <b>結構體參考</b>          | <b>52</b> |
| 概述                    | 52        |
| 定義語法                  | 52        |
| 建構子                   | 52        |
| 欄位存取                  | 53        |
| 欄位賦值                  | 53        |
| 作為函式引數與回傳值使用          | 53        |
| 巢狀結構體                 | 53        |
| 限制與錯誤                 | 53        |
| 列舉型別 (enum)           | 54        |
| <b>測試功能</b>           | <b>55</b> |
| 執行方式                  | 55        |
| 語法                    | 56        |
| 輸出格式                  | 56        |
| 範例                    | 57        |
| 模擬 (Mock)             | 57        |
| 限制事項                  | 58        |
| <b>型別參考</b>           | <b>58</b> |
| 型別一覽                  | 58        |
| 型別標註語法                | 58        |
| 可用型別名稱一覽              | 59        |
| 型別別名                  | 59        |
| 字面量型別                 | 60        |
| 範圍型別                  | 60        |
| none 關鍵字與 Option 型別簡寫 | 61        |
| F-String (字串插值)       | 61        |
| 型別轉換 (as)             | 61        |
| 帶關聯資料的 enum (ADT)     | 62        |
| 泛型 enum               | 62        |
| Error 型別              | 63        |
| union 型別              | 64        |

|                |    |
|----------------|----|
| 型別規則（運算時的型別轉換） | 64 |
| 型別安全性限制        | 65 |

[English](#) | [日本語](#) | [繁體中文](#)

## 字串操作函式參考

針對字串（`str`）的操作函式一覽。所有函式皆可使用 UFCS 記法。

**注意：**所有字串操作皆支援 UTF-8。`len()`、`char_at()`、`substring()`、`find()` 和 `reverse()` 以 Unicode 碼位為單位操作，而非位元組。如需取得位元組長度，請使用 `byte_len()`。

### 函式一覽

#### 搜尋與判定

| 函式                       | 簽名                                          | 說明                                  |
|--------------------------|---------------------------------------------|-------------------------------------|
| <code>contains</code>    | <code>(str, str) → bool</code>              | 回傳是否包含子字串                           |
| <code>starts_with</code> | <code>(str, str) → bool</code>              | 回傳是否以前綴開頭                           |
| <code>ends_with</code>   | <code>(str, str) → bool</code>              | 回傳是否以後綴結尾                           |
| <code>find</code>        | <code>(str, str) → Option&lt;int&gt;</code> | 回傳子字串的字元位置（未找到為 <code>None</code> ） |

#### 擷取與轉換

| 函式                     | 簽名                                 | 說明               |
|------------------------|------------------------------------|------------------|
| <code>substring</code> | <code>(str, int, int) → str</code> | 取得子字串（字元索引）      |
| <code>char_at</code>   | <code>(str, int) → str</code>      | 取得指定位置的 UTF-8 字元 |
| <code>replace</code>   | <code>(str, str, str) → str</code> | 全部取代子字串          |

#### 大小寫

| 函式                    | 簽名                     | 說明           |
|-----------------------|------------------------|--------------|
| <code>to_upper</code> | <code>str → str</code> | 轉換為 ASCII 大寫 |
| <code>to_lower</code> | <code>str → str</code> | 轉換為 ASCII 小寫 |

#### 去除空白

| 函式                      | 簽名                     | 說明      |
|-------------------------|------------------------|---------|
| <code>trim</code>       | <code>str → str</code> | 去除前後的空白 |
| <code>trim_start</code> | <code>str → str</code> | 去除開頭的空白 |
| <code>trim_end</code>   | <code>str → str</code> | 去除結尾的空白 |

#### 生成與加工

| 函式                   | 簽名                            | 說明                     |
|----------------------|-------------------------------|------------------------|
| <code>repeat</code>  | <code>(str, int) → str</code> | 將字串重複 <code>n</code> 次 |
| <code>reverse</code> | <code>str → str</code>        | 反轉字串（UTF-8 感知）         |

| 函式                    | 簽名                     | 說明         |
|-----------------------|------------------------|------------|
| <code>byte_len</code> | <code>str → int</code> | 回傳字串的位元組長度 |

## 分割與連接

| 函式                 | 簽名                                        | 說明      |
|--------------------|-------------------------------------------|---------|
| <code>split</code> | <code>(str, str) → List&lt;str&gt;</code> | 以分隔符號分割 |
| <code>join</code>  | <code>(List&lt;str&gt;, str) → str</code> | 以分隔符號連接 |

## 型別轉換

| 函式                    | 簽名                                    | 說明        |
|-----------------------|---------------------------------------|-----------|
| <code>to_int</code>   | <code>str → int</code>                | 將字串轉換為整數  |
| <code>to_float</code> | <code>str → float</code>              | 將字串轉換為浮點數 |
| <code>to_str</code>   | <code>int/float/bool/str → str</code> | 將☐轉換為字串   |

## contains

簽名：`contains(s: str, sub: str) -> bool`

回傳字串 `s` 中是否包含子字串 `sub`。

```
print(contains("hello", "ell"))    # true
print("hello".contains("xyz"))    # false (UFCS)
```

## starts\_with

簽名：`starts_with(s: str, prefix: str) -> bool`

回傳字串 `s` 是否以 `prefix` 開頭。

```
print(starts_with("hello", "hel")) # true
print("hello".starts_with("world")) # false (UFCS)
```

## ends\_with

簽名：`ends_with(s: str, suffix: str) -> bool`

回傳字串 `s` 是否以 `suffix` 結尾。

```
print(ends_with("hello", "llo"))  # true
print("hello".ends_with("world")) # false (UFCS)
```

## find

簽名: `find(s: str, sub: str) -> Option<int>`

回傳字串 `s` 中子字串 `sub` 首次出現的字元位置。未找到時回傳 `None`。

```
print(find("hello world", "world"))    # Some(6)
print(find("hello", "xyz"))            # None
print("abcdef".find("cd"))              # Some(2) (UFCS)
```

---

## substring

簽名: `substring(s: str, start: int, end: int) -> str`

回傳字串 `s` 從 `start` 到 `end` (不含) 的子字串。索引為字元位置 (UTF-8 感知)。

```
print(substring("hello world", 0, 5))   # hello
print(substring("hello world", 6, 11))  # world
print("abcdef".substring(1, 4))         # bcd (UFCS)
```

---

## char\_at

簽名: `char_at(s: str, i: int) -> str`

回傳字串 `s` 第 `i` 個位置的 UTF-8 字元作為字串。

```
print(char_at("hello", 0))              # h
print("abc".char_at(2))                  # c (UFCS)
```

---

## replace

簽名: `replace(s: str, old: str, new: str) -> str`

回傳將字串 `s` 中所有 `old` 替換為 `new` 後的新字串。

```
print(replace("hello world", "world", "ry"))    # hello ry
print(replace("aaa", "a", "bb"))                # bbbbbb
print("foo bar foo".replace("foo", "baz"))       # baz bar baz (UFCS)
```

---

## to\_upper

簽名: `to_upper(s: str) -> str`

回傳將 ASCII 小寫字母 (a-z) 轉換為大寫後的新字串。

```
print(to_upper("hello"))                 # HELLO
print("Hello World".to_upper())          # HELLO WORLD (UFCS)
```

---

## to\_lower

簽名: `to_lower(s: str) -> str`

回傳將 ASCII 大寫字母 (A-Z) 轉換為小寫後的新字串。

```
print(to_lower("HELLO"))      # hello
print("Hello World".to_lower()) # hello world (UFCS)
```

---

## trim

簽名：trim(s: str) -> str

回傳去除字串前後空白字元（空格、定位字元、換行、回車）後的新字串。

```
print(trim("  hello "))      # hello
print("  hi  ".trim())       # hi (UFCS)
```

---

## trim\_start

簽名：trim\_start(s: str) -> str

回傳去除字串開頭空白字元後的新字串。

```
print(trim_start("  hello ")) # hello
print("  hi".trim_start())    # hi (UFCS)
```

---

## trim\_end

簽名：trim\_end(s: str) -> str

回傳去除字串結尾空白字元後的新字串。

```
print(trim_end("  hello "))  # hello
print("hi  ".trim_end())     # hi (UFCS)
```

---

## repeat

簽名：repeat(s: str, n: int) -> str

回傳將字串 s 重複 n 次後的新字串。

```
print(repeat("ab", 3))       # ababab
print("ha".repeat(3))       # hahaha (UFCS)
```

---

## reverse

簽名：reverse(s: str) -> str

回傳字元順序反轉後的新字串（UTF-8 感知）。

```
print(reverse("hello"))      # olleh
print("abc".reverse())       # cba (UFCS)
```

---

## byte\_len

簽名: `byte_len(s: str) -> int`

回傳字串 `s` 的位元組長度。與回傳 UTF-8 字元數的 `len()` 不同, `byte_len()` 回傳的是位元組數。

```
print(byte_len("hello"))    # 5
print(byte_len("あいう"))   # 9
print(len("あいう"))        # 3 (字元數)
```

---

## split

簽名: `split(s: str, delim: str) -> List<str>`

以分隔符號 `delim` 分割字串 `s`, 回傳 `List<str>`。

```
let parts = split("a,b,c", ",")
print(parts[0])    # a
print(parts[1])    # b
print(parts[2])    # c

for word in "hello world".split(" "):
    print(word)
# hello
# world
```

---

## join

簽名: `join(xs: List<str>, sep: str) -> str`

以分隔符號 `sep` 連接字串串列的元素, 回傳結合後的字串。

```
let parts = ["a", "b", "c"]
print(join(parts, ","))      # a,b,c
print(parts.join("-"))      # a-b-c (UFCS)
```

---

## to\_int

簽名: `to_int(s: str) -> int`

將字串轉換為整數。

```
print(to_int("42"))         # 42
print(to_int("-7"))         # -7
print("123".to_int())       # 123 (UFCS)
```

---

## to\_float

簽名: `to_float(s: str) -> float`

將字串轉換為浮點數。

```
print(to_float("3.14"))     # 3.14
print("2.5".to_float())     # 2.5 (UFCS)
```



## to\_str

簽名: `to_str(v: int | float | bool | str) -> str`

將 `Any` 轉換為字串。

| 型別    | 轉換格式             |
|-------|------------------|
| int   | %ld              |
| float | %g               |
| bool  | "true" / "false" |
| str   | 直接回傳             |

```
print(to_str(42))           # 42
print(to_str(3.14))         # 3.14
print(to_str(true))         # true
print(99.to_str())          # 99 (UFCS)
```

[English](#) | [日本語](#) | [繁體中文](#)

## 內建函式參考

### 函式一覽

#### 核心

| 函式                                                                                            | 說明                                                           |
|-----------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| <code>print(expr)</code><br><code>len(x)</code>                                               | 將 <code>Any</code> 輸出到標準輸出<br>回傳串列、映射、集合的元素數量，或字串的 UTF-8 字元數 |
| <code>range(n)</code> / <code>range(start, end)</code> / <code>range(start, end, step)</code> | 生成整數串列                                                       |
| <code>exit(code)</code><br><code>args()</code>                                                | 以指定的結束碼終止程序<br>以 <code>List&lt;str&gt;</code> 回傳命令列引數        |

#### Option

| 函式                      | 說明                                              |
|-------------------------|-------------------------------------------------|
| <code>Some(expr)</code> | 建構 <code>Option</code> 型別的有 <code>Any</code> 變體 |

#### 集合操作

| 函式                                                                                                                                   | 說明                                                                                     |
|--------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------|
| <code>has_key(map, key)</code><br><code>add(set, value)</code><br><code>remove(set, value)</code>                                    | 回傳映射中是否存在該鍵<br>向集合新增元素（重複則忽略）<br>從集合刪除元素                                               |
| <code>append(list, value)</code> / <code>append!(list, value)</code><br><code>appended(list, value)</code><br><code>pop(list)</code> | 向串列末尾新增元素（就地修改）<br>傳回新增元素後的新串列（非破壞性）<br>移除並回傳串列的最後一個元素（ <code>Option&lt;T&gt;</code> ） |

| 函式                                           | 說明                                              |
|----------------------------------------------|-------------------------------------------------|
| <code>reverse(list)</code>                   | 傳回反轉後的新串列（也適用於字串）                               |
| <code>reverse!(list)</code>                  | 就地反轉串列（破壞性）                                     |
| <code>slice(list, start, end)</code>         | 傳回從 <code>start</code> 到 <code>end</code> 的新子串列 |
| <code>filter(list, pred)</code>              | 傳回僅包含滿足述詞的元素的串列                                 |
| <code>map(list, fn)</code>                   | 傳回將每個元素轉換後的新串列                                  |
| <code>sort(list) / sort(list, comp)</code>   | 傳回排序後的新串列（預設升序）                                 |
| <code>sort!(list) / sort!(list, comp)</code> | 就地排序串列（破壞性）                                     |
| <code>insert(list, i, val)</code>            | 在索引 <code>i</code> 處插入元素                        |
| <code>remove_at(list, i)</code>              | 移除並回傳索引 <code>i</code> 處的元素                     |
| <code>items(map)</code>                      | 回傳 (鍵, 值) 元組的串列                                 |
| <code>remove(map, key)</code>                | 刪除指定鍵的條目                                        |
| <code>get(map, key)</code>                   | 回傳鍵的 <code>Option&lt;V&gt;</code>               |
| <code>get(map, key, default)</code>          | 回傳鍵的 <code>Option</code> ，若不存在則回傳預設值            |
| <code>union(set, set)</code>                 | 回傳兩個集合的聯集                                       |
| <code>intersection(set, set)</code>          | 回傳兩個集合的交集                                       |
| <code>difference(set, set)</code>            | 回傳兩個集合的差集                                       |
| <code>symmetric_difference(set, set)</code>  | 回傳兩個集合的對稱差                                      |
| <code>is_subset(set, set)</code>             | 回傳第一個集合是否為第二個的子集                                |
| <code>is_superset(set, set)</code>           | 回傳第一個集合是否為第二個的超集                                |

## 字串操作

| 函式                                                 | 說明                                          |
|----------------------------------------------------|---------------------------------------------|
| <code>contains(s, sub)</code>                      | 是否包含子字串                                     |
| <code>starts_with(s, prefix)</code>                | 是否以前綴開頭                                     |
| <code>ends_with(s, suffix)</code>                  | 是否以後綴結尾                                     |
| <code>find(s, sub)</code>                          | 子字串的字元位置 ( <code>Option&lt;int&gt;</code> ) |
| <code>byte_len(s)</code>                           | 回傳字串的位元組長度                                  |
| <code>substring(s, start, end)</code>              | 取得子字串                                       |
| <code>char_at(s, i)</code>                         | 取得指定位置的字元                                   |
| <code>replace(s, old, new)</code>                  | 全部取代子字串                                     |
| <code>to_upper(s) / to_lower(s)</code>             | 大小寫轉換                                       |
| <code>trim(s) / trim_start(s) / trim_end(s)</code> | 去除空白                                        |
| <code>repeat(s, n)</code>                          | 將字串重複 <code>n</code> 次                      |
| <code>reverse(s)</code>                            | 反轉字串                                        |
| <code>split(s, delim)</code>                       | 分割字串並回傳串列                                   |
| <code>join(list, sep)</code>                       | 以分隔符號連接串列中的字串                               |
| <code>to_int(s) / to_float(s) / to_str(v)</code>   | 型別轉換                                        |

→ 詳細請參閱 [字串操作函式參考](#)

## print

簽名：`print(expr)`

將 `expr` 輸出到標準輸出。末尾會加上換行。

| 型別            | 輸出格式                      |
|---------------|---------------------------|
| int           | %ld                       |
| float         | %g                        |
| bool          | true / false              |
| str           | %s                        |
| Option (Some) | Some(☐)                   |
| Option (None) | None                      |
| list          | [元素 1, 元素 2, ...]         |
| map           | {鍵 1: ☐ 1, 鍵 2: ☐ 2, ...} |
| set           | {元素 1, 元素 2, ...}         |
| enum          | 變體名稱 (例如: Red)            |

```
print(42)           # 42
print(3.14)         # 3.14
print(true)         # true
print("hello")      # hello
print(Some(1))      # Some(1)
print(None)         # None
print([1, 2, 3])    # [1, 2, 3]
print({"a": 1})     # {a: 1}
print({1, 2, 3})    # {1, 2, 3}
```

錯誤條件：直接傳入結構體或元組會產生編譯錯誤。

## Some

簽名: `Some(expr) -> Option<T>`

建構 Option 型別的有☐變體。

```
let x: Option<int> = Some(42)
print(x)           # Some(42)
```

## len

簽名: `len(x: List<T> | Map<K, V> | Set<T> | str) -> int`

回傳串列、映射、集合的元素數量，或字串的 UTF-8 字元數。如需取得位元組長度，請使用 `byte_len()`。

```
print(len([1, 2, 3]))           # 3
print(len({"a": 1, "b": 2}))    # 2
print(len({1, 2, 3}))           # 3
print(len("hello"))             # 5
print(len("あいう"))            # 3 (UTF-8 字元數)
```

## has\_key

簽名: `has_key(m: Map<K, V>, key: K) -> bool`

回傳映射中是否存在指定的鍵。也可使用 UFCS 記法。

```
let m = {"a": 1, "b": 2}
print(has_key(m, "a"))    # true
print(m.has_key("z"))     # false (UFCS)
```

---

## add

簽名: `add(s: Set<T>, value: T)`

向集合新增元素。若元素已存在則不做任何操作。也可使用 UFCS 記法。

```
let s = {1, 2, 3}
s.add(4)          # UFCS
add(s, 5)         # 一般呼叫
s.add(1)          # 已存在, 因此忽略
print(len(s))     # 5
```

---

## remove

簽名: `remove(s: Set<T>, value: T)`

從集合刪除元素。也可使用 UFCS 記法。

```
let s = {1, 2, 3}
s.remove(2)       # UFCS
print(2 in s)     # false
```

---

## range

簽名: `range(n: int) -> List<int>` / `range(start: int, end: int) -> List<int>` / `range(start: int, end: int, step: int) -> List<int>`

生成整數串列。

| 形式                                   | 生成的☐                                                          |
|--------------------------------------|---------------------------------------------------------------|
| <code>range(n)</code>                | <code>[0, 1, ..., n-1]</code>                                 |
| <code>range(start, end)</code>       | <code>[start, start+1, ..., end-1]</code>                     |
| <code>range(start, end, step)</code> | <code>[start, start+step, start+2*step, ...]</code> (不包含 end) |

- `step > 0` 時, 從 `start` 向 `end` 遞增生成。
- `step < 0` 時, 從 `start` 向 `end` 遞減生成。
- `step == 0` 時, 會產生執行時錯誤。

```
print(range(3))          # [0, 1, 2]
print(range(2, 5))       # [2, 3, 4]
print(range(0, 10, 2))   # [0, 2, 4, 6, 8]
print(range(10, 0, -3))  # [10, 7, 4, 1]
```

```
for i in range(3):
    print(i)
# 0
# 1
# 2
```

---

## exit

簽名：`exit(code: Int)`

以指定的結束碼立即終止程序。`exit()` 之後的程式碼將不會被執行。

```
exit(0)      # 正常終止
exit(1)      # 錯誤終止
```

---

## args

簽名：`args() -> List<String>`

以字串串列的形式回傳傳遞給腳本的命令列引數。不包含直譯器名稱或腳本檔案名稱——僅包含腳本路徑之後的引數。

```
# 執行: ry script.ry hello world
let a = args()
print(len(a))    # 2
print(a[0])      # hello
print(a[1])      # world

for x in args():
    print(x)
```

---

## append

簽名：`append(list: List<T>, value: T)`

向串列末尾新增元素。此為就地修改操作——串列會被直接修改。也可使用 UFCS 記法。

```
var xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

---

## pop

簽名：`pop(list: List<T>) -> Option<T>`

移除並回傳串列的最後一個元素（`Option<T>`）。串列為空時回傳 `None`。也可使用 UFCS 記法。

```
var xs = [1, 2, 3]
let v = xs.pop()
print(v)    # Some(3)
print(xs)   # [1, 2]
```

---

## reverse (串列)

簽名：`reverse(list: List<T>) -> List<T>`

傳回元素順序反轉的新串列。原始串列不會被修改。也適用於字串（請參閱[字串操作](#)）。也可使用 UFCS 記法。

```
let xs = [1, 2, 3]
let ys = reverse(xs)
print(ys)    # [3, 2, 1]
print(xs)    # [1, 2, 3] (未修改)
```

---

## slice

簽名: `slice(list: List<T>, start: int, end: int) -> List<T>`

傳回從 `start` (含) 到 `end` (不含) 的新子串列。索引會被鉗制在有效範圍內 (0 到 `len(list)`)。也可使用 UFCS 記法。

```
let xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (鉗制)
```

---

## filter

簽名: `filter(list: List<T>, pred: fn(T) -> bool) -> List<T>`

傳回僅包含述詞回傳 `true` 的元素的新串列。原始串列不會被修改。也可使用 UFCS 記法。

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.filter((x: int) -> x > 3)
print(ys)    # [4, 5]
print(xs)    # [1, 2, 3, 4, 5] (未修改)
```

---

## map

簽名: `map(list: List<T>, fn: fn(T) -> U) -> List<U>`

傳回將每個元素以給定函式轉換後的新串列。輸出元素型別可以與輸入不同。原始串列不會被修改。也可使用 UFCS 記法。

```
let xs = [1, 2, 3]
let ys = xs.map((x: int) -> x * 2)
print(ys)    # [2, 4, 6]
```

---

## sort

簽名: `sort(list: List<T>) -> List<T>` / `sort(list: List<T>, comp: fn(T, T) -> bool) -> List<T>`

傳回排序後的新串列。預設為升序。可提供自訂比較函式 (第一引數應排在第二引數之前時回傳 `true`)。原始串列不會被修改。也可使用 UFCS 記法。

```
let xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]
```

# 降序排序

```
let desc = xs.sort((a: int, b: int) -> a > b)
print(desc)    # [3, 2, 1]
```

---

## sort!

簽名: `sort!(list: List<T>) / sort!(list: List<T>, comp: fn(T, T) -> bool)`

就地排序串列。排序演算法與 `sort()` 相同，但修改原始串列而非建立新串列。也可使用 UFCS 記法。

```
var xs = [3, 1, 2]
xs.sort!()
print(xs)    # [1, 2, 3]
```

---

## reverse!

簽名: `reverse!(list: List<T>)`

就地反轉串列。也可使用 UFCS 記法。

```
var xs = [1, 2, 3]
xs.reverse!()
print(xs)    # [3, 2, 1]
```

---

## appended

簽名: `appended(list: List<T>, value: T) -> List<T>`

傳回新增元素後的新串列。原始串列不會被修改。也可使用 UFCS 記法。

```
let xs = [1, 2]
let ys = xs.appended(3)
print(xs)    # [1, 2] (未修改)
print(ys)    # [1, 2, 3]
```

---

## append!

簽名: `append!(list: List<T>, value: T)`

`append()` 的別名。就地向串列末尾新增元素。為配合 `!` 命名慣例而提供。

---

## first

簽名: `first(list: List<T>) -> Option<T>`

傳回串列的第一個元素 (`Option<T>`)。串列為空時回傳 `None`。

```
print(first([10, 20, 30]))    # Some(10)
```

---

## last

簽名: `last(list: List<T>) -> Option<T>`

傳回串列的最後一個元素 (`Option<T>`)。串列為空時回傳 `None`。

```
print(last([10, 20, 30]))    # Some(30)
```

---

## get (映射)

簽名: `get(map: Map<K, V>, key: K) -> Option<V>` / `get(map: Map<K, V>, key: K, default: V) -> V`

兩引數形式回傳鍵的 `Option<V>`。三引數形式回傳鍵的 `Option`，若不存在則回傳預設 `Option`。

```
let m = {"a": 1, "b": 2}
print(get(m, "a"))           # Some(1)
print(get(m, "z"))           # None
print(get(m, "z", 0))        # 0
```

[English](#) | [日本語](#) | [繁體中文](#)

## 集合參考 (元組、串列、映射、集合)

### 元組

#### 概述

固定長度、異質型別的 `Option` 組合。以 LLVM literal StructType 實作，是堆疊上的 `Option` 型別。

#### 語法

```
let t = (1, 3.14)
let t: (int, float) = (1, 3.14)
```

#### 型別標註

```
let pair: (int, str) = (42, "hello")
let triple: (int, float, bool) = (1, 2.0, true)
```

#### 元素存取

使用 `.0`、`.1`、`...` 的數 `Option` 索引存取。

```
let t = (10, 3.14)
print(t.0)    # 10
print(t.1)    # 3.14
```

#### 函式回傳 `Option`

```
fn swap(a: int, b: int) -> (int, int):
    return (b, a)
```

```
let result = swap(1, 2)
print(result.0)    # 2
print(result.1)    # 1
```

#### 限制與錯誤

| 限制      | 詳細   |
|---------|------|
| 超出範圍的索引 | 編譯錯誤 |



| 限制                         | 詳細                             |
|----------------------------|--------------------------------|
| 直接將元組傳給 <code>print</code> | 編譯錯誤 ( <code>print</code> 不支援) |

## 串列

### 概述

相同型別的可變長度序列。分配於堆積上。

### 語法

```
let xs = [1, 2, 3]
let xs: List<int> = [1, 2, 3]
```

### 支援的元素型別

`int`, `float`, `bool`, `str`

### 索引存取

```
let xs = [1, 2, 3]
print(xs[0])    # 1
print(xs[2])    # 3
```

### 索引賦

```
let xs = [1, 2, 3]
xs[0] = 99
print(xs[0])    # 99
```

### len

```
let xs = [1, 2, 3]
print(len(xs))  # 3
```

### print

```
let xs = [1, 2, 3]
print(xs)       # [1, 2, 3]
```

### for 走訪

```
let xs = [10, 20, 30]
for x in xs:
    print(x)
# 10
# 20
# 30
```

### append

向串列末尾新增元素。此為就地修改操作。

```
var xs = [1, 2]
xs.append(3)
print(xs)    # [1, 2, 3]
```

### pop

移除並回傳串列的最後一個元素。對空串列呼叫會產生執行時錯誤。

```
var xs = [1, 2, 3]
let v = xs.pop()
print(v)      # 3
print(xs)     # [1, 2]
```

### reverse

傳回元素順序反轉的新串列。原始串列不會被修改。也適用於字串。

```
let xs = [1, 2, 3]
print(reverse(xs))    # [3, 2, 1]
print(xs)              # [1, 2, 3] (未修改)
```

### slice

傳回從 start（含）到 end（不含）的新子串列。索引會被鉗制在有效範圍內。

```
let xs = [1, 2, 3, 4, 5]
print(slice(xs, 1, 3))    # [2, 3]
print(slice(xs, 0, 100))  # [1, 2, 3, 4, 5] (鉗制)
```

### filter

傳回僅包含滿足述詞的元素的新串列。原始串列不會被修改。

```
let xs = [1, 2, 3, 4, 5]
let ys = xs.filter(fn(x: int): x > 3)
print(ys)    # [4, 5]
```

### map

傳回將每個元素以給定函式轉換後的新串列。輸出元素型別可以與輸入不同。原始串列不會被修改。

```
let xs = [1, 2, 3]
let ys = xs.map(fn(x: int): x * 2)
print(ys)    # [2, 4, 6]
```

### sort

傳回排序後的新串列。預設為升序。可提供自訂比較函式。原始串列不會被修改。

```
let xs = [3, 1, 2]
print(xs.sort())    # [1, 2, 3]
```

# 降序排序

```
let desc = xs.sort(fn(a: int, b: int): a > b)
print(desc)    # [3, 2, 1]
```

## filter、map、sort 的鏈接

這些函式傳回新串列，因此可透過 UFCS 進行鏈接。

```
let xs = [5, 3, 1, 4, 2]
let result = xs.filter(fn(x: int): x > 1).map(fn(x: int): x * 10).sort()
print(result)    # [20, 30, 40, 50]
```

## reduce

使用累加函式將串列歸約為單一值，以第一個元素作為初始值。

```
let xs = [1, 2, 3, 4, 5]
let total = reduce(xs, fn(a: int, b: int): a + b)
print(total)    # 15
```

## fold

使用明確的初始值和累加函式將串列折疊為單一值。

```
let xs = [1, 2, 3, 4, 5]
let total = fold(xs, 0, fn(a: int, b: int): a + b)
print(total)    # 15
```

## any

如果至少有一個元素滿足述詞，則傳回 true。

```
let xs = [1, 2, 3, 4, 5]
print(any(xs, fn(x: int): x > 4))    # true
print(any(xs, fn(x: int): x > 9))    # false
```

## all

如果所有元素都滿足述詞，則傳回 true。

```
let xs = [2, 4, 6]
print(all(xs, fn(x: int): x > 0))    # true
print(all(xs, fn(x: int): x > 3))    # false
```

## sum

傳回所有元素的總和。

```
let xs = [1, 2, 3, 4, 5]
print(sum(xs))    # 15
```

## min

傳回最小的元素。

```
let xs = [3, 1, 4, 1, 5]
print(min(xs))    # 1
```

## max

傳回最大的元素。

```
let xs = [3, 1, 4, 1, 5]
print(max(xs))    # 5
```

### first

傳回第一個元素。對空串列呼叫會產生執行時錯誤。

```
let xs = [10, 20, 30]
print(first(xs))    # 10
```

### last

傳回最後一個元素。對空串列呼叫會產生執行時錯誤。

```
let xs = [10, 20, 30]
print(last(xs))    # 30
```

### is\_empty

如果串列沒有元素則傳回 true。

```
let xs = [1, 2, 3]
print(is_empty(xs))    # false
```

### enumerate

傳回 (索引, 元素) 元組的串列。

```
let xs = [10, 20, 30]
let pairs = enumerate(xs)
# pairs = [(0, 10), (1, 20), (2, 30)]

# for 迴圈中的元組解構
for i, x in enumerate(xs):
    print(f"{i}: {x}")    # 0: 10, 1: 20, 2: 30
```

### zip

將兩個串列合併為 (元素 1, 元素 2) 元組的串列。結果長度等於較短的串列。

```
let xs = [1, 2, 3]
let ys = ["a", "b", "c"]
let pairs = zip(xs, ys)
# pairs = [(1, "a"), (2, "b"), (3, "c")]

# for 迴圈中的元組解構
for a, b in zip(xs, ys):
    print(f"{a}: {b}")    # 1: a, 2: b, 3: c
```

### insert

在指定索引處插入元素。該索引及之後的元素向右移動。

```
var xs = [1, 2, 3]
insert(xs, 1, 99)
print(xs)    # [1, 99, 2, 3]
```

### remove\_at

移除並回傳指定索引處的元素。該索引之後的元素向左移動。

```
var xs = [1, 2, 3, 4]
let v = remove_at(xs, 1)
print(v)      # 2
print(xs)     # [1, 3, 4]
```

### remove

從串列中移除第一個符合的指定值。若找不到該值，則不做任何操作。這是一個可變操作。

```
var xs = [1, 2, 3, 2, 4]
remove(xs, 2)
print(xs)  # [1, 3, 2, 4]
```

### distinct

回傳一個移除重複元素的新串列。保持原始順序（保留第一次出現）。原始串列不會被修改。

```
let xs = [1, 2, 3, 2, 1, 4]
print(distinct(xs))  # [1, 2, 3, 4]
print(xs)            # [1, 2, 3, 2, 1, 4] (未變更)
```

### flatten

將巢狀串列（串列的串列）展開一層。回傳新串列。原始串列不會被修改。

```
let xs = [[1, 2], [3, 4]]
print(flatten(xs))  # [1, 2, 3, 4]
print(xs)           # [[1, 2], [3, 4]] (未變更)
```

### 操作複雜度

| 操作                     | 複雜度        |
|------------------------|------------|
| xs[i] 索引存取             | O(1)       |
| append / append!       | 均攤 O(1)    |
| pop                    | O(1)       |
| first、last             | O(1)       |
| insert、remove_at       | O(n)       |
| sort / sort!           | O(n log n) |
| filter、map、reduce、fold | O(n)       |
| reverse / reverse!     | O(n)       |
| distinct               | O(n)       |
| len                    | O(1)       |

### 限制與錯誤

| 限制          | 詳細               |
|-------------|------------------|
| 所有元素必須為相同型別 | 混合不同型別會產生編譯錯誤    |
| 空串列 []      | 無法進行型別推論，會產生編譯錯誤 |
| 超出範圍的存取     | 執行時錯誤 (exit(1))  |

## 映射

### 概述

鍵與值的對應表。分配於堆積上。

### 語法

```
let m = {"a": 1, "b": 2}
let m: Map<str, int> = {"a": 1, "b": 2}
```

### 鍵存取

```
let m = {"a": 1, "b": 2}
print(m["a"])    # 1
```

### 插入與更新

```
let m = {"a": 1}
m["b"] = 2      # 新增
m["a"] = 99     # 更新
```

### len

```
let m = {"a": 1, "b": 2, "c": 3}
print(len(m))   # 3
```

### print

```
let m = {"a": 1, "b": 2}
print(m)        # {a: 1, b: 2}
```

### has\_key

```
let m = {"a": 1, "b": 2}
print(m.has_key("a"))  # true
print(m.has_key("z"))  # false
```

### keys

傳回映射中所有鍵的串列。

```
let m = {"a": 1, "b": 2, "c": 3}
print(keys(m))    # ["a", "b", "c"]
```

### values

傳回映射中所有值的串列。

```
let m = {"a": 1, "b": 2, "c": 3}
print(values(m))  # [1, 2, 3]
```

### items

回傳映射中所有條目的（鍵，值）元組串列。

```
let m = {"a": 1, "b": 2}
let pairs = items(m)
# pairs = [("a", 1), ("b", 2)]
```

## remove (映射)

從映射中刪除指定鍵的條目。若鍵不存在則不做任何操作。

```
let m = {"a": 1, "b": 2}
remove(m, "a")
print(m)    # {b: 2}
```

## get

回傳指定鍵的ⓧ，若鍵不存在則回傳預設ⓧ。

```
let m = {"a": 1, "b": 2}
print(get(m, "a", 0))    # 1
print(get(m, "z", 0))    # 0
```

## merge

回傳一個合併兩個映射的新映射。當鍵重複時，第二個映射的ⓧ優先。原始映射不會被修改。

```
let m1 = {"a": 1, "b": 2}
let m2 = {"b": 99, "c": 3}
let m3 = merge(m1, m2)
print(m3["a"])    # 1
print(m3["b"])    # 99
print(m3["c"])    # 3
```

## 限制與錯誤

| 限制         | 詳細                                                             |
|------------|----------------------------------------------------------------|
| 所有鍵必須為相同型別 | 混合不同型別的鍵會產生編譯錯誤                                                |
| 所有ⓧ必須為相同型別 | 混合不同型別的ⓧ會產生編譯錯誤                                                |
| 空映射        | 需要型別標註 (如 <code>let m: Map&lt;str, int&gt; = {"a": 1}</code> ) |
| 存取不存在的鍵    | 執行時錯誤 (exit(1))                                                |
| 鍵搜尋        | 雜湊表 (平均 O(1))                                                  |
| 容量超過時      | 自動擴展為 2 倍                                                      |

## 集合

### 概述

保持相同型別的元素且不重複的集合。分配於堆積上。

### 語法

```
let s = {1, 2, 3}
let s: Set<int> = {1, 2, 3}
```

### 支援的元素型別

int, float, bool, str

### in 運算子 (歸屬檢査)

```
let s = {1, 2, 3}
print(2 in s)    # true
print(5 in s)    # false
```

### len

```
let s = {1, 2, 3}
print(len(s))    # 3
```

### print

```
let s = {1, 2, 3}
print(s)         # {1, 2, 3}
```

### add (新增元素)

新增重複的元素時會被忽略。

```
let s = {1, 2, 3}
s.add(4)         # 新增
s.add(1)         # 已存在, 因此忽略
print(len(s))    # 4
```

### remove (刪除元素)

```
let s = {1, 2, 3}
s.remove(2)
print(2 in s)    # false
```

### for 走訪

```
let s = {10, 20, 30}
for x in s:
    print(x)
```

### 空集合

空集合需要型別標註。

```
let s: Set<int> = {}
```

### 函式引數

```
fn has_value(s: Set<int>, v: int) -> bool:
    return v in s
```

### union

回傳包含兩個集合所有元素的新集合。

```
let a = {1, 2, 3}
let b = {3, 4, 5}
print(union(a, b))    # {1, 2, 3, 4, 5}
```



## intersection

回傳僅包含兩個集合中都存在的元素的新集合。

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(intersection(a, b))    # {2, 3}
```

## difference

回傳包含在第一個集合中但不在第二個集合中的元素的新集合。

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(difference(a, b))    # {1}
```

## symmetric\_difference

回傳包含在任一集合中但不同時在兩個集合中的元素的新集合。

```
let a = {1, 2, 3}
let b = {2, 3, 4}
print(symetric_difference(a, b))    # {1, 4}
```

## is\_subset

如果第一個集合的所有元素都包含在第二個集合中，則回傳 `true`。

```
let a = {1, 2}
let b = {1, 2, 3}
print(is_subset(a, b))    # true
print(is_subset(b, a))    # false
```

## is\_superset

如果第一個集合包含第二個集合的所有元素，則回傳 `true`。

```
let a = {1, 2, 3}
let b = {1, 2}
print(is_superset(a, b))    # true
print(is_superset(b, a))    # false
```

## 限制與錯誤

| 限制                  | 詳細              |
|---------------------|-----------------|
| 所有元素必須為相同型別         | 混合不同型別會產生編譯錯誤   |
| 空集合 <code>{}</code> | 需要型別標註          |
| 元素搜尋                | 雜湊表（平均 $O(1)$ ） |
| 容量超過時               | 自動擴展為 2 倍       |

[English](#) | [日本語](#) | [繁體中文](#)

## 契約式設計 (Design by Contract)

Ry 支援 Eiffel 風格的契約式設計，包含前置條件（`require`）、後置條件（`ensure`）和結構體不變量（`invariant`）。契約違反時，程式將以 `exit(1)` 終止。

---

## 前置條件 (require)

前置條件在函式進入時檢閱。它們指定函式被正確呼叫所需滿足的條件。

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  var new_balance: int = balance + amount
  return new_balance
```

若前置條件未滿足，程式將輸出以下訊息並終止：

```
Contract violation: require failed in deposit()
```

---

## 後置條件 (ensure)

後置條件在每個 return 之前檢閱。它們指定函式對其回傳值的保證。

### result 關鍵字

在 ensure 區塊中，result 指向回傳值。

```
fn abs(x: int) -> int:
  ensure:
    result >= 0
  if x < 0:
    return -x
  return x
```

### old() 表達式

old(expr) 擷取函式本體執行前表達式的值。適用於比較前後狀態。

```
fn increment(x: int) -> int:
  ensure:
    result == old(x) + 1
  return x + 1
```

---

## 組合範例

```
fn deposit(amount: int, balance: int) -> int:
  require:
    amount > 0
    balance >= 0
  ensure:
    result >= 0
    result == old(balance) + amount
  var new_balance: int = balance + amount
  return new_balance
```

---

## 結構體不變量 (invariant)

不變量是結構體實例必須始終滿足的條件。在以下時機檢查：- 建構時 - 每次欄位賦值後

```
record BankAccount:
    balance: int
    min_balance: int
    invariant:
        balance >= min_balance

let a = BankAccount(100, 0)    # OK: 100 >= 0
a.balance = -1                # Contract violation: invariant failed
```

---

## 規則

- `require` 和 `ensure` 區塊為選用，寫在函式本體之前。
- 同時使用時，`require` 必須在 `ensure` 之前。
- `result` 和 `old()` 只能在 `ensure` 區塊中使用。
- `invariant` 寫在 `record` 定義的末尾，所有欄位宣告之後。
- 所有契約違反以 `exit(1)` 終止程式並輸出診斷訊息。

[English](#) | [日本語](#) | [繁體中文](#)

## 控制流程參考

### if / elif / else

#### 語法

```
if 條件式:
    # then 區塊
elif 條件式:
    # elif 區塊 (可多個)
else:
    # else 區塊 (可省略)
```

#### 條件式的型別

| 型別   | 為 false 的☐ | 為 true 的☐ |
|------|------------|-----------|
| bool | false      | true      |
| int  | 0          | 非 0       |

`float` 和 `str` 無法直接用於條件式。

#### 範例

```
let x = 10

if x > 5:
    print("big")
elif x == 5:
    print("five")
else:
    print("small")
```

## 作用域規則

- `if` / `elif` / `else` 的各個區塊分別擁有獨立的區塊作用域。
- 在區塊內宣告的變數無法從區塊外存取。

```
if true:
    let y = 42
# y 在此處無法存取
```

---

## while

### 語法

```
while 條件式:
    # 迴圈主體
```

當條件式為 `true` 時，重複執行迴圈主體。

### 範例

```
let i = 0
while i < 5:
    print(i)
    i += 1
```

### 搭配 `break` / `continue`

```
let i = 0
while true:
    if i >= 3:
        break
    i += 1
```

---

## for

### 語法

```
# 串列 / 集合走訪
for x in iterable_expr:
    # 各元素依序賦值給 x

# range (從 0 開始)
for i in range(n):
    # i = 0, 1, ..., n-1

# range (指定起始與結束)
for i in range(start, end):
    # i = start, start+1, ..., end-1

# range (指定步長)
for i in range(start, end, step):
    # i = start, start+step, start+2*step, ...
```

## 映射鍵⌘走訪

```
for k, v in map_expr:
    # k 為鍵, v 為各條目的⌘
```

## 元組解構

走訪 2 元素元組的列表（例如 `enumerate()` 或 `zip()` 的回傳⌘）時，可以解構為兩個變數。使用 `_` 捨棄⌘。

```
let xs = [10, 20, 30]

for i, x in enumerate(xs):
    print(f"{i}: {x}")      # 0: 10, 1: 20, 2: 30

for a, b in zip([1, 2], [10, 20]):
    print(a + b)           # 11, 22

for _, x in enumerate(xs):
    print(x)               # 捨棄索引
```

## 範圍運算子 (..)

`..` 運算子建立包含兩端的整數範圍。 `1 .. 5` 產生 `[1, 2, 3, 4, 5]`。

```
for i in 1 .. 5:
    print(i)               # 1 2 3 4 5
```

## 範例

```
let xs = [10, 20, 30]
for x in xs:
    print(x)

let s = {1, 2, 3}
for x in s:
    print(x)

for i in range(5):
    print(i)               # 0 1 2 3 4

for i in range(2, 6):
    print(i)               # 2 3 4 5

for i in range(0, 10, 2):
    print(i)               # 0 2 4 6 8

for i in range(10, 0, -3):
    print(i)               # 10 7 4 1

# 映射走訪
let m = {"a": 1, "b": 2}
for k, v in m:
    print(k)
    print(v)

# 範圍運算子
```

```
for i in 1 .. 3:
    print(i)      # 1 2 3
```

---

## break

- 立即跳出最內層的迴圈（while 或 for）。
- 在迴圈外使用會產生編譯錯誤。

```
for i in range(10):
    if i == 5:
        break      # 在 i == 5 時跳出
    print(i)        # 0 1 2 3 4
```

## 錯誤範例

```
# 在迴圈外使用 break 會產生編譯錯誤
break      # Error: break outside loop
```

---

## continue

- 結束最內層迴圈的當前迭代，跳至下一次迭代。
- 在迴圈外使用會產生編譯錯誤。

```
for i in range(5):
    if i == 2:
        continue   # 跳過 i == 2
    print(i)        # 0 1 3 4
```

---

## ... (Ellipsis)

- 不執行任何操作的語句（no-op）。用作空區塊的佔位符。
- 可在任何區塊中使用：函數主體、if/elif/else、while、for、match case 等。

```
fn not_yet():
    ...

if true:
    ...
else:
    ...
```

---

## match

### 語法

```
match 運算式:
    case 模式:
        # 主體
    case 模式 if 守衛條件:
        # 帶守衛的主體
```

```
case _:
  # 萬用字元（匹配任何☐）
```

## 模式的種類

| 模式                   | 範例                            | 說明                   |
|----------------------|-------------------------------|----------------------|
| 萬用字元                 | <code>_</code>                | 匹配任何☐                |
| 字面☐                  | <code>0, "hello", true</code> | ☐的相等比較               |
| 變數綁定                 | <code>n</code>                | 匹配任何☐並綁定到變數          |
| enum 變體              | <code>Color::Red</code>       | enum 標籤的比較（簡單 enum）  |
| ADT enum 變體          | <code>Shape::Circle(r)</code> | 匹配帶有關聯資料的 enum 變體並綁定 |
| <code>Some(x)</code> | <code>Some(v)</code>          | 當 Option 有☐時，綁定其內容   |
| <code>None</code>    | <code>None</code>             | 當 Option 無☐時         |
| OR 模式                | <code>1 \   2 \   3</code>    | 任一替代方案匹配時即匹配         |

## guard 子句

可以使用 `case` 模式 `if` 條件式：的形式指定守衛條件。只有當模式匹配且守衛條件為真時，該分支才會被執行。

## OR 模式

可以使用 `|` 組合多個模式，任一模式匹配時即匹配。OR 模式中不允許使用變數綁定（`n`、`Some(x)`、`Ok(v)`、`Err(e)`）。

```
match x:
  case 1 | 2 | 3:
    print("small")
  case _:
    print("other")

# enum OR 模式
match color:
  case Color::Red | Color::Blue:
    print("warm or cool")
  case Color::Green:
    print("green")
```

## 窮舉性檢☐

- enum 型別：必須覆蓋所有變體或包含 `_`。OR 模式中的各替代方案會分別計算。
- Option 型別：必須覆蓋 `Some` 和 `None` 或包含 `_`。
- bool 型別：必須覆蓋 `true` 和 `false` 或包含 `_`。
- int / float / str 字面☐：`_` 為必要。
- 帶守衛的分支不計入窮舉性。

## 範例

```
# enum 匹配
enum Color:
  Red
  Green
```

Blue

```
match color:
  case Color::Red:
    print("red")
  case Color::Green:
    print("green")
  case Color::Blue:
    print("blue")

# Option 匹配
let x: Option<int> = Some(42)
match x:
  case Some(v):
    print(v)
  case None:
    print("nothing")

# 字面量匹配
match x:
  case 0:
    print("zero")
  case 1:
    print("one")
  case _:
    print("other")

# guard 子句
match x:
  case n if n > 0:
    print("positive")
  case n if n < 0:
    print("negative")
  case _:
    print("zero")
```

## ADT enum 匹配

當 enum 變體攜帶關聯資料時，使用綁定模式來取出。

```
enum Shape:
  Circle(float)
  Rectangle(float, float)
  Point

let s = Shape::Circle(3.14)
match s:
  case Shape::Circle(r):
    print(r)          # 3.14
  case Shape::Rectangle(w, h):
    print(w)
    print(h)
  case Shape::Point:
    print("point")
```



多欄位變體會按宣告順序將各欄位綁定到不同的名稱。

## 作用域規則

- 各 `case` 分支擁有區塊作用域。
- 透過變數綁定模式 (`n`) 或 `Some(x)` 綁定的變數僅在該分支內有效。

---

## 作用域規則

### 區塊作用域

- `if` / `elif` / `else` / `while` / `for` / `match` 的各區塊擁有區塊作用域。
- 在區塊內宣告的變數會在區塊結束時離開作用域。

```
for i in range(3):  
    let tmp = i * 2  
# tmp 在此處無法存取
```

```
if true:  
    let a = 1  
# a 在此處無法存取
```

### 遮蔽

- 在內層作用域中宣告與外層同名的變數時，在內層作用域內會參照內層的變數。
- 離開內層作用域後會恢復為外層的變數。

```
let x = 10  
if true:  
    let x = 99 # 遮蔽外層的 x  
    print(x)   # 99  
print(x)       # 10
```

[English](#) | [日本語](#) | [繁體中文](#)

## 指令

指令是可以附加到宣告上的編譯時元資料。使用 `@name` 語法。

### 語法

```
@name  
@name(key=value, ...)
```

指令放置在目標宣告之前。可以堆疊多個指令。

### 支援的目標

指令可以套用到以下宣告：

- `fn` - 函式定義
- `record` - 結構體定義
- `let` / `var` - 變數宣告
- `record` 定義內的欄位

## 內建指令

### @deprecated

將宣告標記為已棄用。當已棄用的實體被使用（呼叫、參照或存取）時，會發出編譯時警告。

#### 套用於函式：

```
@deprecated
fn old_function() -> int:
    return 42

print(old_function())    # warning: 'old_function' is deprecated
```

#### 套用於型別：

```
@deprecated
record OldPoint:
    x: int
    y: int

let p = OldPoint(1, 2)  # warning: 'OldPoint' is deprecated
```

#### 套用於變數：

```
@deprecated
let old_value = 99

print(old_value)        # warning: 'old_value' is deprecated
```

#### 套用於欄位：

```
record Config:
    @deprecated
    old_setting: int
    new_setting: int

let c = Config(1, 2)
print(c.old_setting)    # warning: 'Config.old_setting' is deprecated
print(c.new_setting)    # 無警告
```

### @native

宣告由執行環境（內建）提供實作的函式。該函式不能有函式本體。

#### 基本語法：

```
@native
fn contains(s: str, sub: str) -> bool

print(contains("hello world", "world"))  # true
```

#### 運算子多載：

```
@native
fn operator+(a: str, b: str) -> str

print("hello" + " world")  # hello world
```

#### 與 UFCS 搭配使用：

```
@native
```

```
fn to_upper(s: str) -> str

print("hello".to_upper()) # HELLO
```

### 參數數量驗證:

當 `@native` 宣告包含型別簽名時，編譯器會在呼叫處驗證參數數量。支援重載函式（例如：1、2、3 個參數的 `range`），只要任一重載匹配即通過驗證。

### 標準函式庫宣告 (`core/`):

`core/` 目錄包含所有內建函式的 `@native` 宣告，按類別組織：

| 檔案                                                        | 內容                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-----------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>core/builtins.ry</code><br><code>core/str.ry</code> | <code>print</code> , <code>len</code> , <code>range</code> , <code>enumerate</code> , <code>zip</code> , <code>exit</code> , <code>args</code><br><code>contains</code> , <code>starts_with</code> , <code>ends_with</code> , <code>find</code> ,<br><code>substring</code> , <code>char_at</code> , <code>replace</code> , <code>to_upper</code> , <code>to_lower</code> ,<br><code>trim</code> , <code>trim_start</code> , <code>trim_end</code> , <code>repeat</code> , <code>reverse</code> ,<br><code>split</code> , <code>join</code> |
| <code>core/convert.ry</code><br><code>core/list.ry</code> | <code>to_int</code> , <code>to_float</code> , <code>to_str</code><br><code>append</code> , <code>pop</code> , <code>insert</code> , <code>remove_at</code> , <code>slice</code> , <code>distinct</code> ,<br><code>flatten</code> , <code>sort</code> , <code>first</code> , <code>last</code> , <code>is_empty</code>                                                                                                                                                                                                                      |
| <code>core/map.ry</code><br><code>core/set.ry</code>      | <code>keys</code> , <code>values</code> , <code>items</code> , <code>has_key</code> , <code>get</code> , <code>merge</code><br><code>add</code> , <code>remove</code> , <code>union</code> , <code>intersection</code> , <code>difference</code> ,<br><code>symmetric_difference</code> , <code>is_subset</code> , <code>is_superset</code>                                                                                                                                                                                                 |
| <code>core/higher_order.ry</code>                         | <code>filter</code> , <code>map</code> , <code>reduce</code> , <code>fold</code> , <code>any</code> , <code>all</code> , <code>sum</code> , <code>min</code> , <code>max</code>                                                                                                                                                                                                                                                                                                                                                             |

當 `ry` 執行檔附近存在 `core/` 目錄時，這些檔案會作為前導自動載入。前導機制使得內建函式呼叫時的參數數量驗證生效。

**限制事項:** - `@native` 函式不能有本體（簽名後不能加:）。- 加上本體會導致解析錯誤: `@native function must not have a body`。- 宣告的函式必須對應到現有的內建函式，否則在編譯時會發生錯誤。

**未來擴充方向:** - `@native("libfoo.so")` — 綁定外部共享函式庫的 FFI。

### 參數（未來擴充）

指令支援可選的參數語法，為未來擴充做準備:

```
@deprecated(reason="use new_api instead")
fn old_api() -> int:
    return 0
```

目前，參數會被解析但不會被 `@deprecated` 指令使用。

### 注意事項

- 已棄用的實體仍然正常運作，僅會發出警告。
- 警告在使用點發出，不在定義點發出。
- 定義已棄用的實體但不使用它，不會產生警告。
- 未知的指令名稱會導致解析錯誤。
- 在不支援的目標（如 `if`、`while`）上使用指令會導致解析錯誤。

[English](#) | [日本語](#) | [繁體中文](#)

## 錯誤參考

### 錯誤格式

ry 以 Rust 風格的豐富格式顯示編譯錯誤，顯示錯誤的確切位置及原始碼上下文：

```
error: cannot reassign let variable: x
  --> main.ry:5:1
  |
5 | x = 10
  | ^ cannot reassign let variable: x
```

每條錯誤訊息包含：- 錯誤等級和訊息 (error: ...) - 檔案位置 (--> 檔案: 行: 列) - 原始碼行 (相關程式碼) - 插入符號指示器 (^) 指向確切的列位置

### 編譯錯誤一覽

| 錯誤內容                                         | 原因                                                | 範例                                             |
|----------------------------------------------|---------------------------------------------------|------------------------------------------------|
| 賦 $\square$ 給未宣告的變數<br>對 let 變數重新賦 $\square$ | 對未宣告的變數進行賦 $\square$<br>對以 let 宣告的變數重新賦 $\square$ | x = 1 (x 未宣告)<br>let x = 1 $\rightarrow$ x = 2 |
| 重新宣告同名變數                                     | 在同一作用域重新宣告同名變數                                    | let x = 1 $\rightarrow$ let x = 2              |
| 變數型別變更賦 $\square$                            | 對變數賦 $\square$ 不同型別的 $\square$                    | let x = 1 $\rightarrow$ x = 3.14               |
| 型別標註不一致                                      | 宣告時的型別與賦 $\square$ 的型別不同                          | let x: int = 3.14                              |
| 多載回傳型別重複                                     | 定義了引數型別相同但僅回傳型別不同的多載                              | 引數 (int, int) 但回傳型別分別為 int 和 float 的兩個函式       |
| 多載解析失敗                                       | 不存在與引數型別匹配的多載                                     | fn add(a: int, b: int) 卻呼叫 add(1.0, 2.0)       |
| 對位元運算傳入 float                                | 對&、\ 、^、~、<<、>> 傳入 float 型別                       | 3.14 & 1                                       |
| 空串列                                          | 無法從空串列字面 $\square$ [] 推論型別                        | let xs = []                                    |
| 空映射                                          | 無法從空映射字面 $\square$ {} 推論型別                        | let m = {}                                     |
| 元組超出範圍的索引                                    | 存取元組中不存在的索引                                       | let t = (1, 2) $\rightarrow$ t.2               |
| 在迴圈外使用 break/continue                        | 在 for/while 迴圈外使用 break 或 continue                | 在函式頂層使用 break                                  |
| 在區塊內匯入模組                                     | 在函式或條件區塊內使用 from 陳述式                              | 在函式內使用 from math                               |
| 循環匯入                                         | 模組之間互相匯入                                          | a.ry 匯入 b.ry, b.ry 匯入 a.ry                     |
| 相同欄位名重複                                      | 在結構體內定義了兩次相同的欄位名                                  | 在 type T: x: int 中定義了兩個 x                      |
| match 窮舉性不足                                  | match 未覆蓋所有模式                                     | enum 的部分變體未覆蓋、Option 缺少 None、字面 $\square$ 缺少 _ |
| !! 回傳型別不匹配                                   | 在未回傳 (T, Error?) 的函式中使用 !!                        | 在回傳 int 的函式中使用 !!                              |

| 錯誤內容        | 原因                                                 | 範例            |
|-------------|----------------------------------------------------|---------------|
| !! 運算元型別不匹配 | 對非 (T, Error?) <input checked="" type="checkbox"/> | 對普通 int 使用 !! |

### 編譯錯誤範例

```
# 對 let 變數重新賦值 ☒
let x = 10
x = 20 # 錯誤

# 型別變更賦值 ☒
let n = 1
n = "hello" # 錯誤：對 int 型別賦值 ☒ str 型別

# 空串列
let xs = [] # 錯誤：無法推論型別

# 在迴圈外使用 break
break # 錯誤：在迴圈外

# 在區塊內匯入
fn foo():
    from math # 錯誤：僅能在頂層匯入

# 相同欄位名重複
record Bad:
    x: int
    x: float # 錯誤：x 重複
```

### 執行時錯誤一覽

| 錯誤內容      | 原因         | 範例                            |
|-----------|------------|-------------------------------|
| 串列超出範圍的存取 | 串列的索引超出範圍  | let xs = [1, 2, 3]<br>→ xs[5] |
| 映射不存在的鍵存取 | 參照映射中不存在的鍵 | let m = {"a": 1} →<br>m["b"]  |

契約違反 | `require`、`ensure` 或 `invariant` 條件評估為 `false` | 參閱[契約式設計](#) |

所有執行時錯誤都會以 `exit(1)` 結束程序。

### 執行時錯誤範例

```
# 串列超出範圍的存取
let xs = [1, 2, 3]
print(xs[10]) # 執行時錯誤：exit(1)

# 映射不存在的鍵存取
let m = {"a": 1}
print(m["z"]) # 執行時錯誤：exit(1)
```

[English](#) | [日本語](#) | [繁體中文](#)

## 函式參考

### 函式定義語法

fn 函式名 (引數名: 型別, ...) -> 回傳型別:

```
# 主體
return
```

- 引數型別為必要。
- 回傳型別可省略 (省略時為 `Unit`)。
- 主體為縮排的區塊。
- 函式可以定義 `require` (前置條件) 和 `ensure` (後置條件)。參閱 [契約式設計](#)。

**命名慣例：**函式名稱和引數名稱必須使用 `snake_case` (如 `add`、`get_value`、`map_list`)。編譯器會強制執行此慣例。

```
fn add(a: int, b: int) -> int:
    return a + b
```

```
fn greet(name: str):
    print("Hello, " + name)    # 回傳型別為 Unit
```

---

### 引數與回傳的型別

| 項目                | 說明                                                  |
|-------------------|-----------------------------------------------------|
| 引數型別              | 必要。所有引數都需要型別標註                                      |
| 回傳型別              | 可省略。省略時為 <code>Unit</code> (相當於 <code>void</code> ) |
| <code>Unit</code> | 不回傳的函式的回傳型別                                         |

```
fn no_return(x: int):          # 回傳型別 Unit (省略)
    print(x)
```

```
fn get_value() -> int:        # 回傳型別 int
    return 42
```

---

### 遞迴

函式可以呼叫自身。

```
fn factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)
```

---

### 多載

可以定義引數數量或型別不同的同名函式。

## 規則

- 引數的數量或型別不同即可定義同名函式。
- 呼叫時會根據引數的型別和數量選擇適當的函式。
- 僅回傳型別不同的多載是不允許的。

```
fn area(side: int) -> int:
    return side * side
```

```
fn area(w: int, h: int) -> int:
    return w * h
```

```
let a = area(5)      # 25
let b = area(3, 4)   # 12
```

---

## Unit 型別函式

不回傳`Unit`的函式會回傳 `Unit`。回傳型別可以省略。

```
fn log(msg: str):
    print(msg)
```

```
fn log_typed(msg: str) -> Unit:
    print(msg)
```

---

## Lambda 函式

可以就地定義匿名函式。

### 語法

`# 單一運算式（運算式的Unit作為回傳Unit。回傳型別自動推論）`

`fn(引數名: 型別, ...): 運算式`

`# 多行區塊`

`fn(引數名: 型別, ...):`

`# 多個陳述式`

`return Unit`

`# 明確指定回傳型別（可省略）`

`fn(引數名: 型別, ...) -> 回傳型別: 運算式`

### 範例

```
let double = fn(x: int): x * 2
let result = double(5)    # 10
```

```
let add = fn(a: int, b: int): a + b
let sum = add(3, 4)       # 7
```

`# 多行 lambda`

```
let abs = fn(x: int):
    if x < 0:
```

```

    return -x
return x

```

---

## 閉包

Lambda 函式會以`☒`捕獲定義時外層作用域的變數。

```

let base = 10
let add_base = fn(x: int): x + base    # 以☒捕獲 base

base = 99                               # 不影響已捕獲的☒
let r = add_base(5)                    # 15 (使用捕獲時的 base = 10)

```

### 捕獲規則

| 項目        | 內容         |
|-----------|------------|
| 捕獲方式      | ☒捕獲 (複製)   |
| 捕獲時機      | Lambda 定義時 |
| 外層變數修改的影響 | 無 (因為是複製)  |

---

## 函式型別

用於將函式作為`☒`處理的型別。

### 語法

`fn(引數型別 1, 引數型別 2, ...) -> 回傳型別`

### 範例

```

let f: fn(int) -> int = fn(x: int): x * 2
let g: fn(int, int) -> int = fn(a: int, b: int): a + b

fn apply(func: fn(int) -> int, x: int) -> int:
    return func(x)

let result = apply(f, 5)    # 10

```

---

## 高階函式

可以接收函式作為引數，或將函式作為回傳`☒`回傳。

```

fn map_list(xs: List<int>, f: fn(int) -> int) -> List<int>:
    let result: List<int> = []
    for x in xs:
        result += [f(x)]
    return result

let doubled = map_list([1, 2, 3], fn(x: int): x * 2)
# [2, 4, 6]

```



---

## UFCS（統一函式呼叫語法）

可以使用 `a.f(b)` 的形式呼叫 `f(a, b)`。方便用於方法鏈。

### 語法

```
# 一般呼叫
f(a, b)

# UFCS 呼叫（等價）
a.f(b)
```

### 鏈接

```
fn double(x: int) -> int:
    return x * 2

fn add_one(x: int) -> int:
    return x + 1

let result = 5.double().add_one() # double(5) → 10, add_one(10) → 11
```

### 與欄位存取混用

欄位存取（`.field`）和 UFCS（`.method()`）使用相同的點記法，但透過是否有引數來區分。

```
let p = Point(3, 4)
let len = p.x.to_float() # 欄位存取 + UFCS
```

---

## 運算子多載

可以為使用者定義型別定義運算子的行為。

### 語法

```
# 二元運算子（2 個引數）
fn operator<op>(a: 型別, b: 型別) -> 回傳型別:
    ...

# 一元運算子（1 個引數）
fn operator<op>(a: 型別) -> 回傳型別:
    ...
```

### 可多載的運算子

| 種類     | 運算子                                                                                                        |
|--------|------------------------------------------------------------------------------------------------------------|
| 算術（二元） | <code>+</code> <code>-</code> <code>*</code> <code>/</code> <code>%</code> <code>**</code> <code>//</code> |
| 比較（二元） | <code>==</code> <code>!=</code> <code>&lt;</code> <code>&lt;=</code> <code>&gt;</code> <code>&gt;=</code>  |
| 位元（二元） | <code>&amp;</code> <code> </code> <code>^</code> <code>&lt;&lt;</code> <code>&gt;&gt;</code>               |
| 邏輯（二元） | <code>and</code> <code>or</code>                                                                           |
| 一元     | <code>-</code> <code>~</code> <code>not</code>                                                             |

## 二元 / 一元的區別

依引數個數區分。

```
record Vec2:
  x: float
  y: float

# 二元 +
fn operator+(a: Vec2, b: Vec2) -> Vec2:
  return Vec2(a.x + b.x, a.y + b.y)

# 一元 -
fn operator-(v: Vec2) -> Vec2:
  return Vec2(-v.x, -v.y)

# 比較
fn operator==(a: Vec2, b: Vec2) -> bool:
  return a.x == b.x and a.y == b.y

let v1 = Vec2(1.0, 2.0)
let v2 = Vec2(3.0, 4.0)
let v3 = v1 + v2      # Vec2(4.0, 6.0)
let v4 = -v1          # Vec2(-1.0, -2.0)
```

[English](#) | [日本語](#) | [繁體中文](#)

## 運算子參考

### 優先順序表

優先順序數字越小越高（越先被求值）。

| 優先順序 | 運算子                       | 說明             | 結合性 |
|------|---------------------------|----------------|-----|
| 0    | !!                        | 錯誤傳播（後綴）       | 左   |
| 1    | ()                        | 分組             | —   |
| 2    | +x -x ~x                  | 一元正號、負號、位元 NOT | 右   |
| 3    | **                        | 次方             | 右結合 |
| 3.5  | as                        | 型別轉換           | 左   |
| 4    | * / % //                  | 乘法、除法、餘數、整數除法  | 左   |
| 5    | + -                       | 加法、減法          | 左   |
| 6    | << >> >>>                 | 位元移位           | 左   |
| 7    | &                         | 位元 AND         | 左   |
| 8    | ^                         | 位元 XOR         | 左   |
| 9    | \                         | 位元 OR          | 左   |
| 10   | == != < <= > >= in not in | 比較、歸屬          | 左   |
| 11   | not                       | 邏輯 NOT         | 右   |
| 12   | and                       | 邏輯 AND         | 左   |
| 13   | or                        | 邏輯 OR          | 左   |
| 13.5 | ??                        | 空值合併           | 左   |
| 14   | ?:                        | 三元條件           | 右   |

### 算術運算子

| 運算子 | 說明             | 範例                                                                  |
|-----|----------------|---------------------------------------------------------------------|
| +   | 加法 / 字串串接      | $1 + 2 \rightarrow 3$ 、 <code>"a" + "b" \rightarrow "ab"</code>     |
| -   | 減法             | $5 - 3 \rightarrow 2$                                               |
| *   | 乘法 / 字串重複      | $4 * 3 \rightarrow 12$ 、 <code>"ab" * 3 \rightarrow "ababab"</code> |
| /   | 除法 (始終為 float) | $7 / 2 \rightarrow 3.5$                                             |
| //  | 整數除法 (截斷)      | $7 // 2 \rightarrow 3$                                              |
| %   | 餘數             | $7 \% 3 \rightarrow 1$                                              |
| **  | 次方 (始終為 float) | $2 ** 10 \rightarrow 1024.0$                                        |
| -x  | 一元負號           | -5, -3.14                                                           |
| +x  | 一元正號           | +5 (不改變正負號)                                                         |

```
let a = 10 // 3      # 3 (int)
let b = 10 / 3       # 3.3333... (float)
let c = 2 ** 8       # 256.0 (float)
let s = "foo" + "bar" # "foobar"
```

## 比較運算子

全部回傳 bool。

| 運算子 | 說明   |
|-----|------|
| ==  | 等於   |
| !=  | 不等於  |
| <   | 小於   |
| <=  | 小於等於 |
| >   | 大於   |
| >=  | 大於等於 |

- 可用於數值型別 (int / float) 和 bool。
- str 之間以字典序 (位元組順序) 比較。
- in 運算子用於集合、串列、映射的歸屬檢閱 (`x in s`)。
- not in 運算子為 in 的否定 (`x not in s`)。
- 對於映射, in 檢閱鍵是否存在。

```
let x = 3 < 5        # true
let y = "abc" < "abd" # true (字典序)
let s = {1, 2, 3}
let z = 2 in s       # true
let w = 4 not in s   # true
let xs = [1, 2, 3]
let a = 2 in xs      # true (串列線性搜尋)
let m = {"a": 1}
let b = "a" in m     # true (映射鍵搜尋)
```

## 邏輯運算子

| 運算子 | 說明     | 型別                              |
|-----|--------|---------------------------------|
| and | 邏輯 AND | <code>bool × bool → bool</code> |
| or  | 邏輯 OR  | <code>bool × bool → bool</code> |
| not | 邏輯 NOT | <code>bool → bool</code>        |

```
let a = true and false  # false
let b = true or false   # true
let c = not true        # false
```

## 位元運算子

僅可用於 `int` 型別。對 `float` 或 `bool` 使用會產生編譯錯誤。

| 運算子                       | 說明          | 範例                                                   |
|---------------------------|-------------|------------------------------------------------------|
| <code>&amp;</code>        | 位元 AND      | <code>0b1100 &amp; 0b1010 → 0b1000</code>            |
| <code>\ </code>           | 位元 OR       | <code>0b1100 \  0b1010 → 0b1110</code>               |
| <code>^</code>            | 位元 XOR      | <code>0b1100 ^ 0b1010 → 0b0110</code>                |
| <code>~</code>            | 位元 NOT (一元) | <code>~0 → -1</code>                                 |
| <code>&lt;&lt;</code>     | 左移          | <code>1 &lt;&lt; 4 → 16</code>                       |
| <code>&gt;&gt;</code>     | 算術右移        | <code>16 &gt;&gt; 2 → 4</code>                       |
| <code>&gt;&gt;&gt;</code> | 邏輯右移        | <code>-1 &gt;&gt;&gt; 1 → 9223372036854775807</code> |

```
let flags = 0b0001 | 0b0010  # 3
let masked = flags & 0b0011  # 3
let shifted = 1 << 8         # 256
```

## 三元條件運算子

```
let x = condition ? true_value : false_value
```

對 `condition` 進行求值。若為真，回傳 `true_value`；否則回傳 `false_value`。兩個分支必須具有相同的型別。右結合，因此巢狀三元運算子從右向左結合。

```
let x = 3 > 2 ? 10 : 20      # 10
let s = false ? "yes" : "no" # "no"
```

# 巢狀 (右結合)

```
let y = true ? (false ? 1 : 2) : 3  # 2
```

## 範圍運算子

`..` 運算子建立包含兩端的整數範圍。

```
let xs = 1 .. 5  # [1, 2, 3, 4, 5]

for i in 1 .. 3:
    print(i)      # 1 2 3
```

結果是包含從左運算元到右運算元 (兩端皆含) 的所有整數的 `List<int>`。

## 空值合併運算子 (??)

```
let x = option_val ?? default_val
```

如果 `option_val` 為 `Some(v)`，則回傳 `v`。否則回傳 `default_val`。右運算元必須與 `Option` 的內部型別相同。

```
let a: int? = Some(10)
let b: int? = none
```

```
print(a ?? 0)    # 10
print(b ?? 0)    # 0
```

## 複合賦值運算子

更新變數的簡寫形式。x op= y 等價於 x = x op y。

| 運算子     | 等價的運算式     |
|---------|------------|
| x += y  | x = x + y  |
| x -= y  | x = x - y  |
| x *= y  | x = x * y  |
| x /= y  | x = x / y  |
| x %= y  | x = x % y  |
| x //= y | x = x // y |
| x **= y | x = x ** y |
| x &= y  | x = x & y  |
| x  = y  | x = x   y  |
| x ^= y  | x = x ^ y  |
| x <<= y | x = x << y |
| x >>= y | x = x >> y |

```
var x = 10
x += 5    # x = 15
x -= 3    # x = 12
x *= 2    # x = 24
x /= 3    # x = 8
x &= 6    # x = 0
```

## 遞增・遞減運算子

用於將變數增減 1 的後綴運算子。僅可作為陳述式使用。內部分別被轉換為 x = x + 1 和 x = x - 1。

| 運算子 | 等價的運算式    |
|-----|-----------|
| x++ | x = x + 1 |
| x-- | x = x - 1 |

```
var count = 0
count++    # count = 1
count++    # count = 2
count--    # count = 1
```

```
var f = 1.5
f++      # f = 2.5 (int 1 會提升為 float)
```

注意：++ / -- 只能作為陳述式使用，不能在運算式中使用。let 變數不能使用遞增・遞減（不可變性會被強制執行）。

## 運算的型別規則

| 運算                  | 左運算元型別                    | 右運算元型別                 | 結果型別  |
|---------------------|---------------------------|------------------------|-------|
| + - *               | int                       | int                    | int   |
| + - *               | float                     | int / float            | float |
| + - *               | int                       | float                  | float |
| /                   | 任意數☐                      | 任意數☐                   | float |
| //                  | 任意數☐                      | 任意數☐                   | int   |
| **                  | 任意數☐                      | 任意數☐                   | float |
| %                   | int                       | int                    | int   |
| %                   | float 或 int (其中一方為 float) | —                      | float |
| +                   | str                       | str                    | str   |
| == != < <= > >=     | 數☐ / bool / str           | 同型別                    | bool  |
| *                   | str                       | int                    | str   |
| in                  | 任意                        | Set / List / Map<K, V> | bool  |
| not in              | 任意                        | Set / List / Map<K, V> | bool  |
| & \   ^ ~ << >> >>> | int                       | int                    | int   |
| and or not          | bool                      | bool                   | bool  |

## 運算子多載

可以為使用者定義型別定義運算子的行為。

### 語法

# 二元運算子 (2 個引數)

```
fn operator+(a: MyType, b: MyType) -> MyType:
    ...
```

# 一元運算子 (1 個引數)

```
fn operator-(a: MyType) -> MyType:
    ...
```

### 可多載的運算子一覽

| 種類      | 運算子                 |
|---------|---------------------|
| 算術 (二元) | + - * / % ** //     |
| 比較 (二元) | == != < <= > >=     |
| 位元 (二元) | & \   ^ ~ << >> >>> |
| 邏輯 (二元) | and or              |
| 一元      | - ~ not             |

### 二元 / 一元的區別

依引數個數區分。

# 二元 -

```
fn operator-(a: Vec2, b: Vec2) -> Vec2:
    return Vec2(a.x - b.x, a.y - b.y)
```

# 一元 -

```
fn operator-(v: Vec2) -> Vec2:
    return Vec2(-v.x, -v.y)
```

[English](#) | [日本語](#) | [繁體中文](#)

## 套件參考

### 概述

Ry 使用套件系統來組織程式碼。**套件**可以是單一的 .ry 檔案，或是包含多個 .ry 檔案的目錄。使用 from 陳述式匯入套件。

std 套件（標準函式庫）會自動匯入到每個程式中。

---

### 匯入語法

#### 匯入全部定義

```
from math
```

匯入套件內的所有函式與型別。

#### 選擇性匯入

```
from math import add
```

僅匯入指定的定義。

#### 多重選擇性匯入

```
from math import add, sub
```

以逗號分隔選擇性匯入多個定義。

---

### 套件解析

以點號分隔的套件名稱按以下方式解析：

| 匯入陳述式           | 解析結果                             |
|-----------------|----------------------------------|
| from math       | math/ 目錄（套件）或 math.ry 檔案         |
| from utils.math | utils/math/ 目錄或 utils/math.ry 檔案 |
| from std.str    | std/str/ 目錄或 std/str.ry 檔案       |

#### 解析順序

對於每個搜尋路徑：1. **目錄** ({path}/) —若存在，載入目錄內的所有 .ry 檔案（套件）2. **檔案** ({path}.ry) —單一檔案（向後相容）

#### 目錄套件

當套件解析為目錄時：- 目錄內的所有 .ry 檔案會自動載入 - 以\_開頭的檔案會被排除 - 不需要特殊的入口檔案（如\_\_init\_\_.py）- 目錄內檔案中定義的所有函式與型別都會被匯出

```
mypackage/  
  math.ry      # fn add(), fn sub()  
  string.ry    # fn concat()  
  
from mypackage      # 匯入 add, sub, concat  
from mypackage import add  # 僅匯入 add
```

---

## 標準函式庫 (std)

std 套件會自動匯入到每個程式中。提供的功能：- 內建函式 (print, len, range 等) - 字串函式 (contains, find, replace 等) - 型別轉換函式 (to\_int, to\_float, to\_str) - 集合函式 (map, filter, sort 等)

也可以明確匯入特定的定義：

```
from std.str import contains
```

## RY\_HOME

標準函式庫安裝於 \$RY\_HOME/lib/std/。RY\_HOME 的預設值為 ~/.ry。

```
export RY_HOME="$HOME/.ry" # 預設值
```

---

## 搜尋路徑的優先順序

1. 匯入來源檔案所在的目錄
  2. \$RY\_HOME/lib (標準函式庫位置)
  3. 執行檔相對的 lib/ 目錄
  4. RY\_PATH 環境變數中包含的路徑 (以冒號分隔)
- 

## RY\_PATH 環境變數

在 RY\_PATH 中以冒號分隔指定目錄，即可新增至套件搜尋路徑。

```
export RY_PATH="/usr/local/ry/lib:/home/user/ry-packages"
```

---

## 限制

| 限制     | 詳細              |
|--------|-----------------|
| 可使用的位址 | 僅限頂層 (函式或區塊內不可) |
| 重複匯入   | 自動跳過 (不會產生錯誤)   |
| 循環匯入   | 編譯錯誤            |

# 錯誤範例：在區塊內匯入

```
fn main():  
    from math # 錯誤：僅能在頂層匯入
```

# OK：多次匯入相同套件不會產生錯誤

```
from math  
from math # 被跳過
```

---

## 建立套件檔案

### 單一檔案套件

```
# math.ry  
fn add(a: int, b: int) -> int:  
    return a + b
```



```
fn sub(a: int, b: int) -> int:
    return a - b

# main.ry
from math import add, sub

print(add(1, 2))    # 3
print(sub(5, 3))    # 2
```

## 目錄套件

```
mylib/
  math.ry
  string.ry

# main.ry
from mylib import add, concat
```

[English](#) | [日本語](#) | [繁體中文](#)

## 專案管理

### ry init - 專案初始化

將當前目錄初始化為 Ry 專案。

```
ry init
```

### 生成的檔案與目錄

```
my-project/
  ry.toml          # 專案設定檔
  src/
    main.ry        # 進入點（範例程式碼）
```

### 行為

1. 若 ry.toml 已存在則錯誤結束
2. 建立 src/ 目錄（若不存在）
3. 生成 ry.toml (name 為當前目錄名稱)
4. 生成 src/main.ry (若已存在則跳過)

---

### ry self-update - 自我更新

將 ry 本身更新至最新版本。從 GitHub Releases 下載二進位檔並取代目前的執行檔。

```
ry self-update          # 更新至最新穩定版
ry self-update --nightly # 更新至最新 nightly 預先發行版
ry self-update v0.0.1    # 更新至指定版本
```

### 行為

1. 顯示目前版本
2. 根據引數解析更新目標版本
  - 無引數：GitHub 的最新穩定發行版 (/releases/latest)

- `--nightly`：最新的預先發行版
  - 指定版本：指定標籤的發行版
3. 若與目前版本相同，則以 "Already up to date." 結束
  4. 下載二進位檔並取代目前的執行檔

## 注意事項

- 執行需要 `curl` 和 `tar` 指令
- 若因權限不足導致二進位檔取代失敗，會顯示建議使用 `sudo` 的訊息（不會自動執行 `sudo`）
- 下載會先在臨時目錄中進行；但若跨檔案系統的 `cp` 備援操作被中斷，目標二進位檔可能處於不完整狀態

## ry.toml 設定檔

以 TOML 格式記述專案的中繼資料與路徑設定。

```
[project]
name = "my-project"
version = "0.1.0"
entry = "src/main.ry"
```

```
[paths]
src = "src"
```

### [project] 區段

| 鍵                    | 說明              |
|----------------------|-----------------|
| <code>name</code>    | 專案名稱（初始化時為目錄名稱） |
| <code>version</code> | 版本字串            |
| <code>entry</code>   | 作為進入點的原始碼檔案     |

### [paths] 區段

| 鍵                | 說明    |
|------------------|-------|
| <code>src</code> | 原始碼目錄 |

## TOML 子集規格

ry.toml 支援以下 TOML 子集。

- 區段標頭：`[section]`
- 鍵值對：`key = "value"`（僅字串值）
- 註解：從 `#` 到行尾
- 空行會被忽略

[English](#) | [日本語](#) | [繁體中文](#)

## 正規表達式函式參考手冊

正規表達式函式的一覽表。所有函式皆支援 UFCS 記法。模式字串使用標準的正規表達式語法。

注意：Phase 1 中模式以普通字串傳遞。專用的正規表達式字面量語法可能在未來版本中加入。

## 函式一覽

| 函式                          | 簽名                                            | 說明                      |
|-----------------------------|-----------------------------------------------|-------------------------|
| <code>regex_match</code>    | <code>(str, str) -&gt; bool</code>            | 文字全體是否匹配模式              |
| <code>regex_search</code>   | <code>(str, str) -&gt; int</code>             | 傳回第一個匹配的起始位置（找不到時傳回 -1） |
| <code>regex_replace</code>  | <code>(str, str, str) -&gt; str</code>        | 將匹配的部分替換為替換字串           |
| <code>regex_split</code>    | <code>(str, str) -&gt; List&lt;str&gt;</code> | 以模式匹配進行分割               |
| <code>regex_find_all</code> | <code>(str, str) -&gt; List&lt;str&gt;</code> | 傳回所有不重疊的匹配結果            |

## 支援的模式語法

| 語法                  | 說明                | 範例                         |
|---------------------|-------------------|----------------------------|
| <code>abc</code>    | 字面字元              | "hello"                    |
| <code>.</code>      | 任意一個字元（換行除外）      | "a.c" 匹配 "abc"、"aXc"       |
| <code> </code>      | 選擇                | "cat dog" 匹配 "cat" 或 "dog" |
| <code>*</code>      | 零次或多次重複           | "a*" 匹配 ""、"a"、"aaa"       |
| <code>+</code>      | 一次或多次重複           | "a+" 匹配 "a"、"aaa"          |
| <code>?</code>      | 零次或一次             | "a?" 匹配 "" 或 "a"           |
| <code>(...)</code>  | 群組                | "(ab)+" 匹配 "abab"          |
| <code>[abc]</code>  | 字元類別              | "[aeiou]" 匹配母音             |
| <code>[a-z]</code>  | 字元範圍              | "[a-z]+" 匹配小寫單字            |
| <code>[^...]</code> | 否定字元類別            | "[^0-9]" 匹配非數字             |
| <code>^</code>      | 字串開頭錨點            | "^hello"                   |
| <code>\$</code>     | 字串結尾錨點            | "world\$"                  |
| <code>\d</code>     | 數字 [0-9]          | "\d+" 匹配數字                 |
| <code>\D</code>     | 非數字 [^0-9]        |                            |
| <code>\w</code>     | 單字字元 [a-zA-Z0-9_] | "\w+" 匹配識別符                |
| <code>\W</code>     | 非單字字元             |                            |
| <code>\s</code>     | 空白字元              | "\s+" 匹配空格與 Tab            |
| <code>\S</code>     | 非空白字元             |                            |
| <code>\.</code>     | 跳脫特殊字元            | "\." 匹配字面的 .               |

## 使用範例

### `regex_match`

```
print(regex_match("[a-z]+", "hello")) # true
print(regex_match("[0-9]+", "hello")) # false
```

### `regex_search`

```
let pos = regex_search("[0-9]+", "abc123def")
print(pos) # 3
```

### `regex_replace`

```
let s = regex_replace("[0-9]+", "a1b2c3", "X")
print(s) # aXbXcX
```

## regex\_split

```
let parts = regex_split("\\s+", "hello world foo")
print(len(parts)) # 3
print(parts[0])   # hello
```

## regex\_find\_all

```
let matches = regex_find_all("[0-9]+", "a1b23c456")
print(len(matches)) # 3
print(matches[0])   # 1
print(matches[1])   # 23
```

## UFCS 記法

```
# pattern.function(text, ...)
let m = "[a-z]+".regex_match("hello")
print(m) # true
```

[English](#) | [日本語](#) | [繁體中文](#)

# 結構體參考

## 概述

結構體是堆疊上的☐型別。使用 `record` 關鍵字定義。結構體可以使用 `invariant` 子句定義不變量。參閱[契約式設計](#)。

**命名慣例：**結構體名稱必須使用 PascalCase (如 `Point`、`Rectangle`)。欄位名稱必須使用 `snake_case`。編譯器會強制執行這些慣例。

---

## 定義語法

`record` 型別名:  
欄位名: 型別  
欄位名: 型別

## 範例

```
record Point:
  x: int
  y: int

record Rectangle:
  width: float
  height: float
```

---

## 建構子

按照欄位定義順序傳遞引數。不支援具名引數。

```
let p = Point(10, 20)
let r = Rectangle(3.0, 4.5)
```

---

## 欄位存取

使用點記法讀取欄位。

```
let p = Point(10, 20)
print(p.x)    # 10
print(p.y)    # 20
```

---

## 欄位賦

| 變數宣告 | 欄位賦  |
|------|------|
| var  | 可以   |
| let  | 編譯錯誤 |

```
var p = Point(10, 20)
p.x = 100    # OK: var 變數

let q = Point(10, 20)
q.x = 100    # 錯誤: let 變數的欄位不可變更
```

---

## 作為函式引數與回傳使用

```
fn distance(p: Point) -> float:
    return (p.x * p.x + p.y * p.y) as float

fn make_point(x: int, y: int) -> Point:
    return Point(x, y)
```

---

## 巢狀結構體

可以將結構體作為另一個結構體的欄位使用。

```
record Point:
    x: int
    y: int

record Circle:
    center: Point
    radius: float

let c = Circle(Point(0, 0), 1.0)
print(c.center.x)    # 0
```

---

## 限制與錯誤

| 限制      | 詳細   |
|---------|------|
| 相同欄位名重複 | 編譯錯誤 |

| 限制             | 詳細               |
|----------------|------------------|
| let 變數的欄位賦值    | 編譯錯誤             |
| 直接將結構體傳給 print | 編譯錯誤 (print 不支援) |

# 錯誤範例：相同欄位名重複

```
record Bad:
    x: int
    x: int # 錯誤
```

# 錯誤範例：將結構體傳給 print

```
let p = Point(1, 2)
print(p) # 錯誤
```

## 列舉型別 (enum)

### 概述

列舉型別是具名常數的集合。內部以 i64 整數 (0, 1, 2, ...) 表示。

### 定義語法

```
enum 型別名:
    變體名
    變體名
    ...
```

### 範例

```
enum Color:
    Red
    Green
    Blue
```

### 變體存取

使用 :: 運算子存取變體。

```
let c = Color::Red
print(c) # Red
```

### 比較

enum 為整數，因此可以直接使用 == / != 進行比較。

```
print(Color::Red == Color::Red) # true
print(Color::Red != Color::Green) # true
```

### 在 if 陳述式中使用

```
let c = Color::Green
if c == Color::Red:
    print("red")
elif c == Color::Green:
    print("green")
```

```
else:
    print("blue")
```

## 函式引數

型別名稱使用 enum 名稱。

```
fn is_red(c: Color) -> bool:
    return c == Color::Red

print(is_red(Color::Red))    # true
print(is_red(Color::Green))  # false
```

## print

使用 print() 會輸出變體名稱。

```
let c = Color::Blue
print(c)    # Blue
```

## 限制與錯誤

| 限制                          | 詳細                         |
|-----------------------------|----------------------------|
| 變體存取為 EnumName::VariantName | 必須使用 :: 運算子                |
| 變體☐為自動分配                    | 0, 1, 2, ... 的連續編號（無法手動指定） |
| 比較為整數比較                     | 可使用 ==、!=                  |

[English](#) | [日本語](#) | [繁體中文](#)

## 測試功能

Ry 內建 RSpec 風格的測試語法。使用 ry test 子指令執行測試檔案。

### 執行方式

```
ry test          # 自動探索並執行專案內所有 *.test.ry 檔案
ry test test_file.ry # 執行指定的測試檔案
```

結束代碼為 0 表示所有測試通過，1 表示有測試失敗。

### 自動探索模式

不帶引數執行 ry test 時：

1. 搜尋 ry.toml 以找到專案根目錄
2. 在專案根目錄下遞迴探索所有 \*.test.ry 檔案（.git、build、node\_modules 會被跳過）
3. 逐一執行並彙總結果

## 語法

### describe / it

```
describe("說明文字"):
  it("測試案例名稱"):
    # 測試主體
    expect(實際值).to_eq(預期值)
```

- describe 和 it 使用**尾隨區塊語法**: 函式呼叫後加上 : 會將縮排區塊作為 lambda 傳入最後的參數
- describe 區塊內可以撰寫 it 區塊及其他語句 (如變數宣告等)
- 各 it 區塊為獨立的測試案例
- describe / expect 僅能在 `ry test` 中使用 (在一般的 `ry` 執行中會產生編譯錯誤)

### 尾隨區塊語法

任何函式呼叫都可以使用尾隨區塊語法。在 ( ) 後加上 : 會將縮排區塊作為無參數 lambda 傳入最後的參數位置:

# 以下兩者等價:

```
foo("arg"):
  bar()

foo("arg", fn():
  bar()
)
```

### expect / 匹配器

| 匹配器                                      | 說明                          | 支援型別                  |
|------------------------------------------|-----------------------------|-----------------------|
| <code>to_eq(expected)</code>             | 相等比較                        | int, float, bool, str |
| <code>to_not_eq(expected)</code>         | 不相等                         | int, float, bool, str |
| <code>to_be_true()</code>                | 為 true                      | bool                  |
| <code>to_be_false()</code>               | 為 false                     | bool                  |
| <code>to_be_none()</code>                | 為 None                      | Option                |
| <code>to_be_some()</code>                | Option 為 Some               | Option                |
| <code>to_contain(val)</code>             | 容器包含 <code>val</code>       | List, Set, Map, str   |
| <code>to_not_contain(val)</code>         | 容器不包含 <code>val</code>      | List, Set, Map, str   |
| <code>to_be_greater_than(v)</code>       | <code>actual &gt; v</code>  | int, float            |
| <code>to_be_less_than(v)</code>          | <code>actual &lt; v</code>  | int, float            |
| <code>to_be_greater_than_or_eq(v)</code> | <code>actual &gt;= v</code> | int, float            |
| <code>to_be_less_than_or_eq(v)</code>    | <code>actual &lt;= v</code> | int, float            |
| <code>to_have_length(n)</code>           | 長度為 <code>n</code>          | List, Set, Map, str   |
| <code>to_be_empty()</code>               | 長度為 0                       | List, Set, Map, str   |
| <code>to_start_with(prefix)</code>       | 字串以 <code>prefix</code> 開頭  | str                   |
| <code>to_end_with(suffix)</code>         | 字串以 <code>suffix</code> 結尾  | str                   |

## 輸出格式

```
Calculator
+ adds numbers
+ subtracts
- fails test (紅色)
  line 10: expected 3, got 2
```



2 passed, 1 failed

- + 為成功（綠色），- 為失敗（紅色）
  - 失敗時顯示行號與預期✗/實際✗
- 

## 範例

```
describe("Arithmetic"):
  it("adds integers"):
    expect(1 + 2).to_eq(3)

  it("compares strings"):
    expect("hello").to_eq("hello")

  it("checks booleans"):
    expect(3 > 1).to_be_true()

describe("Booleans"):
  it("false check"):
    expect(1 > 2).to_be_false()
```

---

## 模擬 (Mock)

`mock(fn_name, replacement)`

在當前 `it` 區塊中將函式替換為模擬實作。`it` 區塊結束時模擬會自動清除。

```
fn fetch_data() -> str:
  return "real data"

describe("mocking"):
  it("replaces function"):
    mock(fetch_data, fn(): "fake")
    expect(fetch_data()).to_eq("fake")

  it("auto-restores"):
    expect(fetch_data()).to_eq("real data")
```

- 第一個參數為函式名稱（識別符，非字串）
- 第二個參數為替換用 `lambda`
- 替換函式必須與原始函式具有相同的參數型別和回傳型別
- `it` 區塊結束時模擬會自動還原

`verify(fn_name)`

回傳模擬函式被呼叫的次數（`int`）。

```
describe("verify"):
  it("counts calls"):
    mock(fetch_data, fn(): "fake")
    fetch_data()
    fetch_data()
    expect(verify(fetch_data)).to_eq(2)
```

## 模擬的限制事項

- 不支援多載函式的模擬
- 不支援使用捕獲閉包進行模擬（僅支援純 lambda）
- 不支援 `@native fn` 的模擬

## 限制事項

- 不支援 `describe` 的巢狀
- 不支援 `before_each` / `after_each`

[English](#) | [日本語](#) | [繁體中文](#)

## 型別參考

### 型別一覽

| 型別                                                                                      | 內部表示                                                                    | 字面量範例                                                                                  | 說明                                                                                |
|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------|----------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|
| <code>int</code><br><code>byte</code>                                                   | <code>i64</code><br><code>i8</code>                                     | <code>42, -7, 0xFF, 0b1010</code><br>(無專用字面量)                                          | 64 位元有號整數<br>無號 8 位元整數<br>(0-255)。透過型別標註<br><code>let b: byte = 42</code> 使用      |
| <code>float</code><br><code>bool</code><br><code>str</code>                             | <code>f64</code><br><code>!l</code><br><code>ptr</code>                 | <code>3.14, 0.5</code><br><code>true, false</code><br><code>"hello", "", "a\nb"</code> | 64 位元浮點數<br>布林<br>字串（堆積上的不可變位<br>元組序列）                                            |
| <code>Unit</code>                                                                       | <code>void</code>                                                       | (無回傳)                                                                                  | 省略回傳型別時的隱式<br>回傳型別                                                                |
| <code>Option&lt;T&gt;</code><br>( <code>T1, T2, ...</code> )                            | <code>{ !l, T }</code><br><code>LLVM StructType</code><br>(literal)     | <code>Some(42), None</code><br>( <code>1, 3.14</code> )                                | 可能存在字面量的型別<br>元組型別                                                                |
| <code>List&lt;T&gt;</code><br><code>Map&lt;K, V&gt;</code><br><code>Set&lt;T&gt;</code> | <code>ptr</code> (堆積)<br><code>ptr</code> (堆積)<br><code>ptr</code> (堆積) | <code>[1, 2, 3]</code><br><code>{"a": 1}</code><br><code>{1, 2, 3}</code>              | 動態陣列<br>雜湊映射<br>不重複的集合                                                            |
| <code>fn(T1, T2) -&gt; R</code><br>使用者定義型別                                              | <code>ptr</code> (函式指標)<br><code>LLVM StructType</code><br>(named)      | <code>fn(x: int): x * 2</code><br><code>record Point: ...</code>                       | 函式型別<br>以 <code>record</code> 關鍵字定義的<br>結構體                                       |
| <code>enum</code>                                                                       | <code>i64</code> / 標籤聯合                                                 | <code>Color::Red,</code><br><code>Shape::Circle(3.14)</code>                           | 以 <code>enum</code> 關鍵字定義的列<br>舉型別（支援關聯資料）                                        |
| <code>Error</code>                                                                      | <code>{ ptr, i64 }</code>                                               | <code>Error("msg"),</code><br><code>Error("msg", 404)</code>                           | 內建錯誤型別                                                                            |
| <code>T1   T2</code>                                                                    | <code>{ i64, [N x i8] }</code>                                          | <code>int   str</code>                                                                 | union 型別（可持有多種<br>型別之一）                                                           |
| <code>int</code> 字面量<br><code>str</code> 字面量<br>範圍                                      | <code>i64</code><br><code>ptr</code><br><code>i64</code>                | <code>42, 0   1</code><br><code>"N"   "S"</code><br><code>1..12, -10..10</code>        | <code>int</code> 字面量型別（限制）<br><code>str</code> 字面量型別（限制）<br>範圍型別（包含兩端的整<br>數範圍限制） |

## 型別標註語法

宣告變數時可以明確指定型別。當型別可推論時可以省略。

```

let x: int = 42
let b: byte = 255
let f: float = 3.14
let s: str = "hello"
let b: bool = true
let opt: Option<int> = Some(10)
let t: (int, float) = (1, 3.14)
let xs: List<int> = [1, 2, 3]
let m: Map<str, int> = {"a": 1}
let s: Set<int> = {1, 2, 3}
let fn_val: fn(int) -> int = fn(x: int): x * 2
let u: int | str = 42

```

## 可用型別名稱一覽

| 型別名稱             | 備註                             |
|------------------|--------------------------------|
| int              | 內建純量型別                         |
| byte             | 內建純量型別（無號 0-255）               |
| float            | 內建純量型別                         |
| bool             | 內建純量型別                         |
| str              | 內建字串型別                         |
| Unit             | 無回傳函式的回傳型別                     |
| Option<T>        | 泛型型別（T 為任意型別）                  |
| (T1, T2, ...)    | 元組型別（元素數量和型別組合任意）              |
| List<T>          | 泛型動態陣列型別                       |
| Map<K, V>        | 泛型雜湊映射型別                       |
| Set<T>           | 泛型集合型別                         |
| fn(T1, ...) -> R | 函式型別                           |
| Error            | 內建錯誤型別（message: str、code: int） |
| T1 \  T2 \  ...  | union 型別（以 \  分隔的多個型別之一）       |
| 使用者定義型別名稱        | 以 record 或 enum 關鍵字宣告的型別       |
| int 字面量型別        | 以 int 字面量限制（例：42、0 \  1）       |
| str 字面量型別        | 以字串字面量限制（例："N" \  "S"）         |
| 範圍型別             | 以整數範圍限制（例：1..12、-10..10）       |

## 型別別名

type 關鍵字為現有型別建立新的名稱。別名與原始型別完全互換。

```

type Meters = float
type StringList = List<str>

```

```

let d: Meters = 3.14
let names: StringList = ["Alice", "Bob"]

```

**命名慣例：**型別別名名稱必須使用 PascalCase（如 Meters、StringList）。編譯器會強制執行此慣例。

型別別名也可以用於函式型別、字面量型別和範圍型別：

```

type Callback = fn(int, int) -> int

let add: Callback = fn(a: int, b: int): a + b
print(add(3, 4))    # 7

```

```

type Month = 1..12
type Direction = "N" | "S" | "E" | "W"
type Digit = 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

let m: Month = 6
let d: Direction = "N"
let n: Digit = 5

```

## 字面量型別

字面量型別將變數的範圍限制為特定的常數。對於常數，在編譯時進行約束檢查；對於動態，在執行時進行約束檢查。

### int 字面量型別

```

let x: 42 = 42           # 單一字面量型別
let y: 0 | 1 = 0         # int 字面量的 union
var z: 0 | 1 = 0
z = 1                   # OK
# z = 2                 # 編譯錯誤（常數）或執行時錯誤（動態）

```

### str 字面量型別

```

let dir: "N" | "S" | "E" | "W" = "N"
# let bad: "N" | "S" = "X"      # 編譯錯誤

```

### 約束檢查

- 編譯時：當賦值為常數（ConstantInt 或字串字面量）時，在編譯時檢查，違反時產生編譯錯誤
- 執行時：當為動態（如函式回傳）時，在執行時檢查，違反時程式以錯誤退出

## 範圍型別

範圍型別將整數變數的範圍限制在連續的範圍內（包含兩端）。

```

let month: 1..12 = 6      # OK
# let bad: 1..12 = 0      # 編譯錯誤：超出範圍
# let bad: 1..12 = 13     # 編譯錯誤：超出範圍

let t: -10..10 = -5       # 支援負數範圍

```

### 使用 var 重新賦值（執行時檢查）

```

var x: 1..12 = 6
x = 12           # OK
# x = dynamic_value() # 執行時檢查：超出範圍則錯誤退出

```

### 在函式參數中使用

```

fn set_month(m: 1..12) -> int:
    return m

set_month(6)           # OK
# set_month(13)        # 編譯錯誤（常數引數）

```

---

## none 關鍵字與 Option 型別簡寫

none 關鍵字表示 Option 型別的❏不存在，等同於 None。

T? 語法是 Option<T> 的簡寫。

```
let x: int? = 42      # 等同於 Option<int>
let y: int? = none    # 等同於 None

fn find(xs: List<int>, val: int) -> int?:
  for x in xs:
    if x == val:
      return Some(x)
  return none
```

---

## F-String (字串插❏)

使用 f"... " 語法進行字串插❏。{} 內的表達式會被求❏並轉換為字串。

```
let name = "world"
print(f"Hello {name}")      # Hello world

let a = 1
let b = 2
print(f"{a} + {b} = {a + b}")  # 1 + 2 = 3
```

## 插❏中支援的型別

{ } 內可使用求❏結果為 int、float、bool 或 str 的任意表達式。

## 跳脫序列

| 序列          | 輸出        |
|-------------|-----------|
| {{          | { (字面大括號) |
| }}          | } (字面大括號) |
| \n \t \\ \" | 與普通字串相同   |

```
print(f"{{braces}}")      # {braces}
```

## 型別轉換 (as)

使用 as 關鍵字進行明確的型別轉換。

```
let x = 42 as float      # 42.0
let y = 3.14 as int      # 3
let z = 1 as bool        # true
let s = 42 as str        # "42"
let b = 255 as byte      # byte ❏ 255
```

## 支援的轉換

| 來源                 | 目標    | 行為                  |
|--------------------|-------|---------------------|
| int                | float | SIToFP              |
| float              | int   | 截斷 (FPToSI)         |
| int                | bool  | 0 → false、非零 → true |
| bool               | int   | false → 0、true → 1  |
| int / float / bool | str   | 字串表示                |
| int                | byte  | 截斷 (低 8 位元)         |
| byte               | int   | 零擴展                 |

不支援的轉換 (例如 `str as int`) 會產生編譯錯誤。字串轉數☐請使用 `to_int()` / `to_float()`。

## 帶關聯資料的 enum (ADT)

在變體名稱後面加上括號並指定型別，enum 變體就可以攜帶關聯資料。不帶括號的變體仍然是單純的標籤。

```
enum Shape:
    Circle(float)
    Rectangle(float, float)
    Point
```

### 建構子

使用 `EnumName::Variant(value)` 語法建立帶有資料的變體。

```
let c = Shape::Circle(3.14)
let r = Shape::Rectangle(4.0, 5.0)
let p = Shape::Point
```

### 帶綁定的模式匹配

使用 `case EnumName::Variant(binding):` 形式取出關聯資料。

```
match c:
    case Shape::Circle(r):
        print(r)           # 3.14
    case Shape::Rectangle(w, h):
        print(w)
        print(h)
    case Shape::Point:
        print("point")
```

### 內部表示

ADT enum 以標籤聯合的形式儲存：`{ i64 tag, [N x i8] data }`，N 的大小足以容納最大變體的酬載。

## 泛型 enum

enum 可以使用角括號語法 `<T>` 帶型別參數，使相同的 enum 結構可以持有不同型別的酬載。

```
enum MyOption<T>:
    MySome(T)
    MyNone
```

## 使用方式

當編譯器無法推論型別時，需提供具體的型別引數來實例化。

```
let a = MyOption<int>::MySome(42)
let b = MyOption<int>::MyNone
```

```
match a:
  case MyOption::MySome(v):
    print(v)          # 42
  case MyOption::MyNone:
    print("none")
```

---

## Error 型別

用於錯誤處理的內建型別。Error 具有兩個欄位：message (str) 和 code (int)。

```
let e = Error("something went wrong")      # code 預設為 0
let e2 = Error("not found", 404)           # 明確指定 code

print(e.message)    # something went wrong
print(e2.code)      # 404
print(e2)            # Error: not found (code: 404)
```

## 錯誤處理慣例

可能失敗的函式回傳 (T, Error?) 元組：

```
fn divide(a: int, b: int) -> (int, Error?):
  if b == 0:
    return (0, Some(Error("division by zero")))
  return (a // b, none)

let val, err = divide(10, 2)
match err:
  case Some(e):
    print(e.message)
  case None:
    print(val)          # 5
```

## !! 運算子 (錯誤傳播)

!! 後綴運算子從 (T, Error?) 元組中取出 $\Box$ 。如果錯誤存在，會將其傳播給外層函式。

```
fn read_file(path: str) -> (str, Error?):
  if path == "":
    return ("", Some(Error("empty path")))
  return ("content", none)

fn process() -> (str, Error?):
  let data = read_file("test.txt")!!    # 如果有錯誤則傳播
  return (data, none)
```

外層函式也必須回傳 (X, Error?) 才能使用 !!。

## 內部表示

Error 以 { ptr message, i64 code } 表示。

## union 型別

可以使用 | 宣告可能持有多種型別的變數。

```
let x: int | str = 42
x = "hello"      # 可重新賦值 (union 中的任一型別)
print(x)         # hello
```

## 在函式引數與回傳值中的使用

```
fn show(x: int | str) -> int:
    print(x)
    return 0

fn get_val(flag: bool) -> int | str:
    if flag:
        return 42
    return "hello"
```

## 內部表示

union 型別以 { i64 tag, [N x i8] data } 表示。tag 表示各組成型別的索引（按字母順序排序後），data 是最大組成型別大小的位元組陣列。

## 限制

- 賦值不屬於 union 的型別會產生編譯錯誤
- int | str 和 str | int 是相同的型別（會被正規化）
- 使用 print() 輸出 union 值時，會根據執行時的 tag 以適當的型別顯示

## 型別規則（運算時的型別轉換）

| 運算              | 左運算元        | 右運算元                     | 結果型別  | 備註                      |
|-----------------|-------------|--------------------------|-------|-------------------------|
| + - *           | int         | int                      | int   |                         |
| + - *           | byte        | byte 或 int               | int   | byte 在運算時以 ZExt 提升為 int |
| + - *           | float 或 int | float 或 int（其中一方為 float） | float | 隱式 float 提升             |
| /               | 任意數值        | 任意數值                     | float | 始終為 float               |
| //              | 任意數值        | 任意數值                     | int   | float 輸入會截斷轉換           |
| **              | 任意數值        | 任意數值                     | float | 使用 libm pow             |
| %               | int         | int                      | int   |                         |
| %               | float 或 int | float 或 int（其中一方為 float） | float |                         |
| +               | str         | str                      | str   | 字串串接                    |
| == != < <= > >= | str         | str                      | bool  | 字典序比較                   |
| == != < <= > >= | 數值或 bool    | 數值或 bool                 | bool  |                         |
| in              | 任意          | Set                      | bool  | 元素是否包含在集合中              |
| & \   ^ ~ << >> | int         | int                      | int   | 對 float 會產生錯誤           |



跳脫序列 (str 字面␣內)

| 序列              | 意義   |
|-----------------|------|
| <code>\n</code> | 換行   |
| <code>\t</code> | 定位字元 |
| <code>\\</code> | 反斜線  |
| <code>\"</code> | 雙引號  |
| <code>\0</code> | 空字元  |

## 型別安全性限制

- **沒有隱式型別轉換** — 混合使用 `int` 和 `float` 時會發生 `float` 提升，但除此之外不存在隱式轉換。`byte` 在運算時會自動提升為 `int` (ZExt)。僅在型別標註 `let b: byte = 42` 時允許從 `int` 字面␣到 `byte` 的窄化轉換。
- **變數型別在宣告時固定** — 一旦以 `int` 宣告的變數，就無法重新賦␣為 `float`。
- **位元運算僅限 `int`** — 對 `float` 或 `bool` 使用位元運算會產生編譯錯誤。
- **`bool` 以外的型別也可用於條件式** — `if` 的條件式可使用 `int` (`0 = false`、非 `0 = true`) 等 `bool` 以外的型別。